

```

diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/README
--- /tmp/orig/toKen-exchange-main/README.md      2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/README.md      2026-09-24 23:26:46.540202733 +0000
@@ -27,10 +27,30 @@
- Sanctioned-address screening: sends and swaps are checked against a bundled
  OFAC sanctions list before they go out (see "Sanctioned-address screening"
  below).
-- Live prices: a "Live prices" screen shows current USD prices (and 24h
- change) for 18 widely-tracked cryptocurrencies, and the main screen shows
- roughly what your connected account's balance is worth in USD -- both
- powered by CoinGecko's free public API (see "Live prices" below).
+- Live prices: a "Live prices" screen has a Crypto tab (18 core coins plus ~
+ more, searchable, with 24h change and a star-to-pin watchlist) and a
+ Currencies tab (what 1 unit of each of 42 national currencies is worth
+ in your display currency). The main screen shows roughly what your connected
+ account's balance is worth. Settings -> Display currency offers 42 fiat
+ currencies (USD, EUR, GBP, JPY, CHF, CNY, MXN, NGN, ...) and 5 crypto
+ denominations (BTC, ETH, BNB, SOL, XRP). All powered by CoinGecko's free
+ public API (see "Live prices" below). Extra coins whose CoinGecko id isn't
+ returned are quietly left out rather than shown as "n/a".
+ Tapping a coin opens an in-app coin screen (price chart with 24H/7D/1M/1Y
+ ranges, market stats, a short description) with a Swap for <coin>
+ button. Swap is only offered when the coin is the selected network's native
+ coin or a token the user has added (matched by symbol) -- the wallet never
+ guesses token contract addresses -- and the Swap screen's From list shows
+ only what the user actually holds.
+
+- Optional accounts (website only): a username + password account syncs just
+ three display settings across devices -- currency, language and watchlist.
+ It never stores wallet keys, addresses, balances or the recovery phrase, so
+ losing an account (or a breach of this server) can never cost anyone their
+ funds. Off by default; turn on with `ACCOUNTS_ENABLED=1` once a persistent
+ volume is mounted for the SQLite file (see accounts-api.js for details).
+ Not available in the Chrome extension yet (different origin, no shared
+ session) -- the entry point stays hidden there automatically.
- Buy crypto: a "Buy" button opens MoonPay's hosted on-ramp widget in a new
  tab so you can purchase crypto with a card or bank transfer and send it to
  your wallet address (see "Buy crypto (MoonPay)" below).
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/accou
--- /tmp/orig/toKen-exchange-main/accounts-api.js      1970-01-01 00:00:00.00000
+++ toKen-exchange-main/accounts-api.js 2026-09-19 17:05:32.632572898 +0000
@@ -0,0 +1,445 @@
+// accounts-api.js
+// Optional username/password accounts for the WEBSITE, used to sync a few

```

```

+// harmless display settings (currency, language, watchlist) between devices.
+// It is deliberately NOT a wallet backup: no keys, recovery phrases,
+// addresses, balances or contacts are ever accepted or stored here, and the
+// settings endpoint rejects anything outside a tiny whitelist. Losing an
+// account (or having this server breached) can therefore never cost anyone
+// their funds -- the recovery phrase on the person's own device stays the
+// only thing that controls them.
+//
+// HOW LOGIN WORKS
+// The browser never sends the raw password. It derives a 256-bit "auth key"
+// from (username, password) with PBKDF2 and sends only that. The server
+// stores scrypt(authKey, per-user random salt). So even someone who reads
+// the database gets nothing that unlocks a wallet or reveals the password
+// in one step, and a server log that captures a login request doesn't
+// capture the password itself.
+//
+// SESSIONS
+// A random 32-byte token in an HttpOnly, SameSite=Lax cookie (Secure when
+// served over https). Only its SHA-256 is stored server-side. Thirty-day
+// lifetime; logout and account deletion revoke it immediately.
+//
+// CSRF
+// Every state-changing call must be JSON and carry X-TM-Requested-With
+// (a custom header a cross-site form can't send), and any Origin header
+// present must match this site's own host. There are no CORS headers here:
+// the API is same-origin only. (The Chrome extension can't use it yet --
+// that would need bearer-token CORS support, planned for later.)
+//
+// SWITCHED OFF BY DEFAULT
+// Nothing here runs unless ACCOUNTS_ENABLED=1 AND the database opens. Set
+// ACCOUNTS_DB_PATH (or DATA_DIR) to a path on a persistent volume first --
+// on Railway the container filesystem is wiped on every redeploy.
+//
+// STORAGE
+// Node's built-in SQLite (node:sqlite, Node 22.13+): no extra packages.
+// If the runtime doesn't have it, accounts stay disabled and the rest of
+// the site is unaffected.
+
+"use strict";
+
+const crypto = require("crypto");
+const fs = require("fs");
+const path = require("path");
+const { promisify } = require("util");
+const scrypt = promisify(crypto.scrypt);
+
+let DatabaseSync = null;

```

```

+try {
+  ({ DatabaseSync } = require("node:sqlite"));
+} catch (e) {
+  DatabaseSync = null;
+}
+
+const DATA_DIR = process.env.DATA_DIR || path.join(__dirname, "data");
+const DB_PATH = process.env.ACCOUNTS_DB_PATH || path.join(DATA_DIR, "accounts.db");
+
+const SESSION_COOKIE = "tm_session";
+const SESSION_TTL_MS = 30 * 24 * 60 * 60 * 1000;
+const MAX_BODY_BYTES = 16 * 1024;
+const MAX_SETTINGS_BYTES = 8 * 1024;
+const USERNAME_RE = /^[a-z0-9_]{3,24}$/;
+const RESERVED = new Set(["admin", "administrator", "root", "support", "help", "
+const AUTH_KEY_RE = /^[0-9a-f]{64}$/;
+const SCRYPT_PARAMS = { N: 16384, r: 8, p: 1 };
+
+// ---- Database -----
+let db = null;
+let dbTried = false;
+
+function getDb() {
+  if (db || dbTried) return db;
+  dbTried = true;
+  if (process.env.ACCOUNTS_ENABLED !== "1") return null;
+  if (!DatabaseSync) {
+    console.warn("[accounts] ACCOUNTS_ENABLED=1 but this Node has no node:sqlite");
+    return null;
+  }
+  try {
+    fs.mkdirSync(path.dirname(DB_PATH), { recursive: true });
+    const d = new DatabaseSync(DB_PATH);
+    d.exec("PRAGMA journal_mode = WAL; PRAGMA foreign_keys = ON; PRAGMA busy_tim
+    d.exec(`
+      CREATE TABLE IF NOT EXISTS users (
+        id INTEGER PRIMARY KEY AUTOINCREMENT,
+        username TEXT NOT NULL UNIQUE,
+        salt BLOB NOT NULL,
+        hash BLOB NOT NULL,
+        created_at INTEGER NOT NULL,
+        settings TEXT,
+        settings_updated_at INTEGER
+      );
+      CREATE TABLE IF NOT EXISTS sessions (
+        token_hash TEXT PRIMARY KEY,
+        user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,

```

```

+         created_at INTEGER NOT NULL,
+         expires_at INTEGER NOT NULL
+     );
+     CREATE INDEX IF NOT EXISTS idx_sessions_user ON sessions(user_id);
+ `);
+     db = d;
+     console.log("[accounts] Enabled. Database at", DB_PATH);
+ } catch (e) {
+     console.error("[accounts] Could not open database; accounts stay off:", e &&
+     db = null;
+ }
+ return db;
+}
+
+function isEnabled() {
+ return !!getDb();
+}
+
+// ---- Small helpers -----
+function sendJson(res, status, obj, extraHeaders) {
+ const body = JSON.stringify(obj);
+ res.writeHead(status, Object.assign({
+     "Content-Type": "application/json; charset=utf-8",
+     "Content-Length": Buffer.byteLength(body),
+     "Cache-Control": "no-store",
+ }, extraHeaders || {}));
+ res.end(body);
+}
+
+function fail(res, status, code, extraHeaders) {
+ sendJson(res, status, { error: code }, extraHeaders);
+}
+
+// Railway (and most hosts) append the real client address as the LAST
+// X-Forwarded-For entry; earlier entries can be forged by the client.
+function clientIp(req) {
+ const xff = String(req.headers["x-forwarded-for"] || "").split(",").map((s) =>
+ return xff.length ? xff[xff.length - 1] : (req.socket && req.socket.remoteAddr
+}
+
+function isHttps(req) {
+ return String(req.headers["x-forwarded-proto"] || "").split(",")[0].trim() ===
+}
+
+function parseCookies(req) {
+ const out = {};
+ String(req.headers.cookie || "").split(";").forEach((part) => {

```

```

+   const i = part.indexOf("=");
+   if (i > 0) out[part.slice(0, i).trim()] = part.slice(i + 1).trim();
+ });
+ return out;
+}
+
+function sessionCookie(req, token, maxAgeSec) {
+  const parts = [`${SESSION_COOKIE}=${token}`, "Path=/", "HttpOnly", "SameSite=L
+  if (isHttps(req)) parts.push("Secure");
+  return parts.join("; ");
+}
+
+// ---- Rate limiting (in memory; fine for a single instance) -----
+const buckets = new Map(); // "name:key" -> [timestamps]
+
+function allow(name, key, limit, windowMs) {
+  const k = name + ":" + key;
+  const now = Date.now();
+  const arr = (buckets.get(k) || []).filter((t) => now - t < windowMs);
+  if (arr.length >= limit) {
+    buckets.set(k, arr);
+    return false;
+  }
+  arr.push(now);
+  buckets.set(k, arr);
+  return true;
+}
+
+const pruneTimer = setInterval(() => {
+  const now = Date.now();
+  for (const [k, arr] of buckets) {
+    const fresh = arr.filter((t) => now - t < 60 * 60 * 1000);
+    if (fresh.length) buckets.set(k, fresh);
+    else buckets.delete(k);
+  }
+}, 10 * 60 * 1000);
+if (pruneTimer.unref) pruneTimer.unref();
+
+// ---- Request parsing & guards -----
+function readJson(req) {
+  return new Promise((resolve, reject) => {
+    let size = 0;
+    const chunks = [];
+    req.on("data", (c) => {
+      size += c.length;
+      if (size > MAX_BODY_BYTES) {
+        reject(Object.assign(new Error("too_large"), { code: "too_large" }));

```

```

+         req.destroy();
+         return;
+     }
+     chunks.push(c);
+ });
+ req.on("end", () => {
+     try {
+         const text = Buffer.concat(chunks).toString("utf8");
+         resolve(text ? JSON.parse(text) : {});
+     } catch (e) {
+         reject(Object.assign(new Error("bad_request"), { code: "bad_request" }))
+     }
+ });
+ req.on("error", () => reject(Object.assign(new Error("bad_request"), { code:
+ }));
+
+
+// Returns an error code string if the request fails the CSRF checks.
+function csrfProblem(req) {
+  if (req.headers["x-tm-requested-with"] !== "web") return "csrf";
+  if (!String(req.headers["content-type"] || "").toLowerCase().startsWith("appli
+  const origin = req.headers.origin;
+  if (origin) {
+    let host = "";
+    try { host = new URL(origin).host; } catch (e) { return "csrf"; }
+    const mine = String(req.headers["x-forwarded-host"] || req.headers.host || "
+    if (host !== mine) return "csrf";
+  }
+  return null;
+}
+
+
+// ---- Credentials -----
+function validUsername(u) {
+  return typeof u === "string" && USERNAME_RE.test(u) && !RESERVED.has(u);
+}
+
+
+async function hashAuthKey(authKeyHex, salt) {
+  return scrypt(Buffer.from(authKeyHex, "hex"), salt, 32, SCRYPT_PARAMS);
+}
+
+
+const DUMMY_SALT = crypto.randomBytes(16);
+const DUMMY_HASH = crypto.randomBytes(32);
+
+
+// ---- Sessions -----
+function sha256Hex(s) {
+  return crypto.createHash("sha256").update(s).digest("hex");
+}

```

```

+
+function createSession(userId) {
+  const token = crypto.randomBytes(32).toString("base64url");
+  const now = Date.now();
+  getDb().prepare("INSERT INTO sessions (token_hash, user_id, created_at, expire
+    .run(sha256Hex(token), userId, now, now + SESSION_TTL_MS);
+  return token;
+}
+
+function tokenFromRequest(req) {
+  const c = parseCookies(req)[SESSION_COOKIE];
+  if (c) return c;
+  const auth = String(req.headers.authorization || "");
+  return auth.startsWith("Bearer ") ? auth.slice(7).trim() : null;
+}
+
+function sessionUser(req) {
+  const token = tokenFromRequest(req);
+  if (!token || token.length > 100) return null;
+  const row = getDb().prepare(
+    "SELECT u.id AS id, u.username AS username, u.settings AS settings, u.settin
+    FROM sessions s JOIN users u ON u.id = s.user_id WHERE s.token_hash = ?"
+  ).get(sha256Hex(token));
+  if (!row) return null;
+  if (row.expiresAt < Date.now()) {
+    getDb().prepare("DELETE FROM sessions WHERE token_hash = ?").run(sha256Hex(t
+    return null;
+  }
+  return row;
+}
+
+// ---- Settings whitelist -----
+// Only these display preferences sync. Security-relevant settings (auto-lock,
+// send delay) are deliberately excluded so an account can never weaken the
+// protection on another device.
+function normalizeSettings(input) {
+  if (!input || typeof input !== "object" || Array.isArray(input)) return null;
+  const out = {};
+  if (input.currency !== undefined) {
+    if (typeof input.currency !== "string" || !/^[a-z]{2,5}$/.test(input.currenc
+    out.currency = input.currency;
+  }
+  if (input.language !== undefined) {
+    if (typeof input.language !== "string" || !/^[a-z]{2}(-[A-Za-z]{2,4})?$/.tes
+    out.language = input.language;
+  }
+  if (input.watchlist !== undefined) {

```

```

+   if (!Array.isArray(input.watchlist) || input.watchlist.length > 200) return
+   const seen = new Set();
+   for (const s of input.watchlist) {
+     if (typeof s !== "string" || !/^[A-Za-z0-9]{1,12}$/ .test(s)) return null;
+     seen.add(s.toUpperCase());
+   }
+   out.watchlist = Array.from(seen);
+ }
+ const json = JSON.stringify(out);
+ if (Buffer.byteLength(json) > MAX_SETTINGS_BYTES) return null;
+ return out;
+}
+
+function parseStoredSettings(text) {
+  if (!text) return null;
+  try { return JSON.parse(text); } catch (e) { return null; }
+}
+
+// ---- Route handlers -----
+async function handleRegister(req, res) {
+  if (!allow("register-ip", clientIp(req), 5, 60 * 60 * 1000)) return fail(res,
+  const body = await readJson(req);
+  const username = typeof body.username === "string" ? body.username.trim().toLo
+  if (!validUsername(username) || typeof body.authKey !== "string" || !AUTH_KEY_
+    return fail(res, 400, "bad_request");
+  }
+  const d = getDb();
+  if (d.prepare("SELECT 1 FROM users WHERE username = ?").get(username)) return
+  const salt = crypto.randomBytes(16);
+  const hash = await hashAuthKey(body.authKey, salt);
+  let userId;
+  try {
+    const r = d.prepare("INSERT INTO users (username, salt, hash, created_at) VA
+    userId = Number(r.lastInsertRowid);
+  } catch (e) {
+    return fail(res, 409, "username_taken"); // lost a race for the same name
+  }
+  const token = createSession(userId);
+  sendJson(res, 201, { username, settings: null, settingsUpdatedAt: null }, { "S
+}
+
+async function handleLogin(req, res) {
+  const ip = clientIp(req);
+  if (!allow("login-ip", ip, 20, 15 * 60 * 1000)) return fail(res, 429, "rate_li
+  const body = await readJson(req);
+  const username = typeof body.username === "string" ? body.username.trim().toLo
+  if (!validUsername(username) || typeof body.authKey !== "string" || !AUTH_KEY_

```

```

+   return fail(res, 401, "bad_credentials");
+ }
+ if (!allow("login-user", username, 8, 15 * 60 * 1000)) return fail(res, 429, "
+ const d = getDb();
+ const user = d.prepare("SELECT id, username, salt, hash, settings, settings_up
+ // Do the same amount of work whether or not the user exists, so response
+ // time doesn't reveal which usernames are registered.
+ const salt = user ? Buffer.from(user.salt) : DUMMY_SALT;
+ const expected = user ? Buffer.from(user.hash) : DUMMY_HASH;
+ const got = await hashAuthKey(body.authKey, salt);
+ if (!user || !crypto.timingSafeEqual(got, expected)) return fail(res, 401, "ba
+ d.prepare("DELETE FROM sessions WHERE expires_at < ?").run(Date.now());
+ const token = createSession(user.id);
+ sendJson(res, 200, {
+   username: user.username,
+   settings: parseStoredSettings(user.settings),
+   settingsUpdatedAt: user.settingsUpdatedAt || null,
+ }, { "Set-Cookie": sessionCookie(req, token, SESSION_TTL_MS / 1000) });
+}
+
+function handleLogout(req, res) {
+  const token = tokenFromRequest(req);
+  if (token) getDb().prepare("DELETE FROM sessions WHERE token_hash = ?").run(sh
+  sendJson(res, 200, { ok: true }, { "Set-Cookie": sessionCookie(req, "", 0) });
+}
+
+function handleMe(req, res) {
+  const user = sessionUser(req);
+  if (!user) return fail(res, 401, "unauthorized");
+  sendJson(res, 200, {
+    username: user.username,
+    settings: parseStoredSettings(user.settings),
+    settingsUpdatedAt: user.settingsUpdatedAt || null,
+  });
+}
+
+async function handleSettingsPut(req, res) {
+  const user = sessionUser(req);
+  if (!user) return fail(res, 401, "unauthorized");
+  if (!allow("settings-user", String(user.id), 60, 60 * 1000)) return fail(res,
+  const body = await readJson(req);
+  const settings = normalizeSettings(body.settings);
+  if (!settings) return fail(res, 400, "bad_request");
+  const now = Date.now();
+  getDb().prepare("UPDATE users SET settings = ?, settings_updated_at = ? WHERE
+  sendJson(res, 200, { settings, settingsUpdatedAt: now });
+}

```

```

+
+async function handleDelete(req, res) {
+  const user = sessionUser(req);
+  if (!user) return fail(res, 401, "unauthorized");
+  if (!allow("delete-user", String(user.id), 5, 15 * 60 * 1000)) return fail(res
+  const body = await readJson(req);
+  if (typeof body.authKey !== "string" || !AUTH_KEY_RE.test(body.authKey)) retur
+  const row = getDb().prepare("SELECT salt, hash FROM users WHERE id = ?").get(u
+  const got = await hashAuthKey(body.authKey, Buffer.from(row.salt));
+  if (!crypto.timingSafeEqual(got, Buffer.from(row.hash))) return fail(res, 401,
+  getDb().prepare("DELETE FROM users WHERE id = ?").run(user.id); // sessions go
+  sendJson(res, 200, { ok: true }, { "Set-Cookie": sessionCookie(req, "", 0) });
+}
+
+// Returns true if the request was handled here; false to fall through to the
+// static file server.
+function handleAccountsApi(req, res) {
+  const url = (req.url || "").split("?")[0];
+  if (!url.startsWith("/api/account/")) return false;
+
+  const route = url.slice("/api/account/".length);
+
+  if (route === "status" && req.method === "GET") {
+    sendJson(res, 200, { enabled: isEnabled() });
+    return true;
+  }
+
+  if (!isEnabled()) {
+    fail(res, 503, "unavailable");
+    return true;
+  }
+
+  const routes = {
+    "POST register": handleRegister,
+    "POST login": handleLogin,
+    "POST logout": handleLogout,
+    "GET me": handleMe,
+    "PUT settings": handleSettingsPut,
+    "POST delete": handleDelete,
+  };
+
+  const handler = routes[req.method + " " + route];
+  if (!handler) {
+    fail(res, 404, "not_found");
+    return true;
+  }
+
+  if (req.method !== "GET") {

```

```

+   const problem = csrfProblem(req);
+   if (problem) {
+     fail(res, 403, problem);
+     return true;
+   }
+ }
+
+ Promise.resolve()
+   .then(() => handler(req, res))
+   .catch((e) => {
+     if (res.headersSent) return;
+     if (e && e.code === "too_large") return fail(res, 413, "too_large");
+     if (e && e.code === "bad_request") return fail(res, 400, "bad_request");
+     console.error("[accounts] Unexpected error:", e && e.message);
+     fail(res, 500, "server_error");
+   });
+ return true;
+}
+
+module.exports = { handleAccountsApi, isEnabled, normalizeSettings };
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/app.c
--- /tmp/orig/toKen-exchange-main/app.css      2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/app.css 2026-09-19 17:06:30.286716351 +0000
@@ -1239,3 +1239,129 @@
     body.tm-web-wallet { padding: 0; }
     body.tm-web-wallet #app { border-radius: 0; min-height: 100vh; border: none; }
   }
+
+/* Prices screen: Crypto / Currencies tabs + search */
+.prices-tabs {
+  display: flex; gap: 6px; margin: 10px 0 8px;
+  background: var(--bg-inset); border: 1px solid var(--line);
+  border-radius: 10px; padding: 4px;
+}
+.prices-tab {
+  flex: 1; background: transparent; border: 0; border-radius: 7px;
+  color: var(--text-muted); font: inherit; font-weight: 600; font-size: 13px;
+  padding: 7px 10px; cursor: pointer;
+}
+.prices-tab:hover { color: var(--text); }
+.prices-tab.active { background: var(--gold); color: var(--gold-ink); }
+.prices-tab:focus-visible { outline: 2px solid var(--gold-bright); outline-offse
+.prices-search {
+  width: 100%; box-sizing: border-box; margin: 0 0 8px;
+  background: var(--bg-inset); border: 1px solid var(--line); border-radius: 8px
+  color: var(--text); font: inherit; font-size: 13px; padding: 9px 12px;
+}

```

```

+.prices-search:focus { outline: none; border-color: var(--gold); }
+#prices-tab-note { margin: 0 0 8px; font-size: 12px; }
+
+/* Prices screen polish: symbol stacked under the name, price with a 24h
+   change pill stacked on the right, star in its own fixed column so every
+   row lines up regardless of how long the price text is. */
+.price-left { flex: 1; min-width: 0; }
+.price-id { display: flex; flex-direction: column; min-width: 0; line-height: 1.
+.price-id .price-name { overflow: hidden; text-overflow: ellipsis; white-space:
+.price-row .price-id .price-symbol,
+html[dir="rtl"] .price-row .price-id .price-symbol { margin: 0; font-size: 11px;
+.price-right { flex-shrink: 0; gap: 6px; }
+.price-quote { display: flex; flex-direction: column; align-items: flex-end; gap
+.price-quote .price-change,
+html[dir="rtl"] .price-quote .price-change {
+  margin: 0; padding: 1px 7px; border-radius: 999px;
+  font-size: 10.5px; font-weight: 600; line-height: 1.5;
+}
+.price-quote .price-change.up { background: rgba(127, 201, 154, 0.14); }
+.price-quote .price-change.down { background: rgba(226, 104, 90, 0.14); }
+.price-right .star-btn {
+  display: flex; align-items: center; justify-content: center;
+  width: 26px; height: 32px; padding: 0; font-size: 16px;
+}
+.prices-search {
+  padding-inline-start: 34px;
+  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000
+  background-repeat: no-repeat; background-position: 11px center; background-siz
+}
+html[dir="rtl"] .prices-search { background-position: right 11px center; }
+.prices-tab.active { box-shadow: 0 1px 6px rgba(0, 0, 0, 0.35); }
+/* Keep numbers and +/- signs reading correctly inside right-to-left layouts. */
+.price-quote .price-change, .price-quote .price-usd { direction: ltr; unicode-bi
+
+/* Coin rows are links to the coin's CoinGecko page (star stays a button). */
+.price-link {
+  display: flex; align-items: center; justify-content: space-between;
+  flex: 1; min-width: 0; gap: 10px; color: inherit; text-decoration: none;
+  cursor: pointer;
+}
+.price-link:hover .price-name { text-decoration: underline; text-underline-offse
+.price-link:focus-visible { outline: 2px solid var(--gold-bright); outline-offse
+.price-link { margin-inline-end: 6px; }
+
+/* ---- Coin detail screen ---- */
+.coin-head { display: flex; align-items: center; gap: 12px; margin-bottom: 6px;
+.coin-head .coin-icon .token-icon, .coin-head .coin-icon img { width: 40px; heig

```

```

+.coin-title { flex: 1; min-width: 0; }
+.coin-title h2 { margin: 0; line-height: 1.15; }
+.coin-rank {
+  font-size: 11px; font-weight: 700; padding: 3px 8px; border-radius: 999px;
+  background: var(--bg-inset); border: 1px solid var(--line); color: var(--text-
+}
+.coin-price-block { display: flex; align-items: center; gap: 10px; margin: 6px 0
+.coin-price { font-size: 28px; font-weight: 700; color: var(--gold-bright); dire
+.coin-price-block .price-change { font-size: 12px; padding: 2px 8px; border-radi
+.coin-price-block .price-change.up { background: rgba(127, 201, 154, 0.14); }
+.coin-price-block .price-change.down { background: rgba(226, 104, 90, 0.14); }
+.coin-chart { height: 130px; margin: 6px 0 2px; color: var(--text-muted); touch-
+.coin-chart.up { color: #7fc99a; }
+.coin-chart.down { color: #e2685a; }
+.coin-chart svg { width: 100%; height: 100%; display: block; }
+.coin-chart-msg { display: flex; align-items: center; justify-content: center; h
+.coin-chart-readout { min-height: 18px; margin: 2px 0 8px; direction: ltr; unico
+.coin-ranges {
+  display: flex; gap: 6px; margin: 0 0 14px;
+  background: var(--bg-inset); border: 1px solid var(--line); border-radius: 10p
+}
+.coin-range {
+  flex: 1; background: transparent; border: 0; border-radius: 7px; color: var(--
+  font: inherit; font-weight: 600; font-size: 12px; padding: 6px 4px; cursor: po
+}
+.coin-range.active { background: var(--gold); color: var(--gold-ink); }
+.coin-swap-card {
+  background: var(--bg-inset); border: 1px solid var(--line); border-radius: 12p
+  padding: 12px; margin: 0 0 16px;
+}
+.coin-swap-card p { margin: 0 0 8px; }
+.coin-swap-card button { width: 100%; }
+.coin-swap-card button:disabled { opacity: 0.5; cursor: not-allowed; }
+.coin-stats { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; margin-bo
+.coin-stat {
+  background: var(--bg-inset); border: 1px solid var(--line); border-radius: 10p
+  padding: 8px 10px; min-width: 0;
+}
+.coin-stat .k { display: block; font-size: 11px; color: var(--text-muted); margi
+.coin-stat .v { display: block; font-size: 13.5px; font-weight: 600; overflow: h
+/* Price rows now open the coin screen, so they are buttons, not anchors. */
+button.price-link { background: none; border: 0; padding: 0; font: inherit; text
+
+/* ---- Account screen ---- */
+.account-tabs {
+  display: flex; gap: 6px; margin: 10px 0 12px;
+  background: var(--bg-inset); border: 1px solid var(--line);

```

```

+ border-radius: 10px; padding: 4px;
+}
+.account-tab {
+ flex: 1; background: transparent; border: 0; border-radius: 7px;
+ color: var(--text-muted); font: inherit; font-weight: 600; font-size: 13px;
+ padding: 7px 10px; cursor: pointer;
+}
+.account-tab.active { background: var(--gold); color: var(--gold-ink); box-shado
+.account-tab:focus-visible { outline: 2px solid var(--gold-bright); outline-offs
+#screen-account label { display: block; margin: 10px 0 4px; }
+#account-signed-in-as { font-weight: 700; color: var(--gold-bright); margin: 6px
+#account-signed-in .danger, #account-signed-out .primary { width: 100%; margin-t
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/app.j
--- /tmp/orig/toKen-exchange-main/app.js      2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/app.js  2026-09-19 17:07:06.109971830 +0000
@@ -38,7 +38,7 @@
    }

    function formatCurrency(amount) {
-   return `${currencySymbol()}${amount.toLocaleString(undefined, { minimumFractio
+   return TM_PRICES.formatMoney(amount, currentCurrency);
    }

    // ----- PORTFOLIO TO
@@ -980,6 +980,7 @@
    if (watchlistSymbols.has(key)) watchlistSymbols.delete(key);
    else watchlistSymbols.add(key);
    chrome.storage.local.set({ [TM_WATCHLIST_KEY]: Array.from(watchlistSymbols) })
+   scheduleAccountPush();
  }

  // Stable sort (guaranteed by the spec for Array#sort) -- starred coins move
@@ -994,7 +995,7 @@
  const priceText =
    c.price == null
      ? TM_I18N.t("prices.naText")
-     : `${currencySymbol()}${c.price.toLocaleString(undefined, { minimumFractio
+     : TM_PRICES.formatMoney(c.price, currentCurrency, { price: true });
  let changeHtml = "";
  if (typeof c.change24h === "number") {
    const cls = c.change24h >= 0 ? "up" : "down";
@@ -1002,10 +1003,16 @@
    changeHtml = `<span class="price-change ${cls}">${sign}${c.change24h.toFixed
  }
  const starred = isWatchlisted(c.symbol);
+ // The whole row (logo, name, price) opens the in-app coin screen; the
+ // star is a separate button so pinning a coin doesn't navigate away.

```

```

+   const linkLabel = TM_I18N.t("prices.viewCoin", { name: c.name });
+   row.innerHTML = `
-     <span class="price-left">${tokenIconHtml(c.symbol, c.image)}<span><span clas
-     <span class="price-right"><button type="button" class="star-btn ${starred ?
+     <button type="button" class="price-link" aria-label="${linkLabel}" title="${{
+     <button type="button" class="star-btn ${starred ? "starred" : ""}" aria-labe
+   `;
+   row.querySelector(".price-link").addEventListener("click", () => {
+     openCoinDetail(c, $("screen-prices").classList.contains("hidden") ? "screen-
+   });
+   row.querySelector(".star-btn").addEventListener("click", (e) => {
+     e.stopPropagation();
+     toggleWatchlist(c.symbol);
@@ -1014,15 +1021,67 @@
+   return row;
+ }

+// Prices screen state: two tabs (crypto coins / fiat currencies) sharing one
+// search box. Data is fetched once per refresh and filtered client-side as
+// the person types, so searching never triggers another CoinGecko request.
+let pricesTab = "crypto";
+let pricesBoardData = [];
+let pricesRatesData = [];
+
+function renderCurrencyRow(r) {
+  const row = document.createElement("div");
+  row.className = "price-row";
+  row.innerHTML = `
+    <span class="price-left">${tokenIconHtml(r.code)}<span class="price-id"><spa
+    <span class="price-right"><span class="price-quote"><span class="price-usd">
+  `;
+  return row;
+}
+
+function renderPricesList() {
+  const list = $("prices-list");
+  const q = ($("prices-search").value || "").trim().toLowerCase();
+  list.innerHTML = "";
+  let count = 0;
+  if (pricesTab === "crypto") {
+    sortByWatchlist(pricesBoardData).forEach((c) => {
+      if (q && !(c.name.toLowerCase().includes(q) || c.symbol.toLowerCase().incl
+      list.appendChild(renderPriceRow(c));
+      count++;
+    });
+  } else {
+    pricesRatesData.forEach((r) => {

```

```

+     if (q && !(r.name.toLowerCase().includes(q) || r.code.toLowerCase().includ
+     list.appendChild(renderCurrencyRow(r));
+     count++;
+   });
+ }
+ if (!count) {
+   const p = document.createElement("p");
+   p.className = "muted";
+   p.textContent = TM_I18N.t("prices.noResults");
+   list.appendChild(p);
+ }
+}
+
+function setPricesTab(tab) {
+  pricesTab = tab;
+  document.querySelectorAll(".prices-tab").forEach((b) => b.classList.toggle("ac
+  $("prices-tab-note").classList.toggle("hidden", tab !== "currencies");
+  refreshPrices();
+}
+
+document.querySelectorAll(".prices-tab").forEach((b) => b.addEventListener("clie
+$(".prices-search").addEventListener("input", renderPricesList);
+
+  async function refreshPrices() {
+    hideError("prices-error");
+    $("prices-status").innerHTML = coinSpinnerHtml(TM_I18N.t("prices.loading"));
+    $("prices-status").classList.remove("hidden");
+    try {
+      -   const board = sortByWatchlist(await TM_PRICES.getPriceBoard(currentCurrency)
+      -   const list = $(".prices-list");
+      -   list.innerHTML = "";
+      -   board.forEach((c) => list.appendChild(renderPriceRow(c)));
+      +   if (pricesTab === "crypto") pricesBoardData = await TM_PRICES.getPriceBoard(
+      +   else pricesRatesData = await TM_PRICES.getFiatRates(currentCurrency);
+      +   renderPricesList();
+      +   $("prices-status").classList.add("hidden");
+    } catch (e) {
+      +   $("prices-status").classList.add("hidden");
+    }
+  }
+}
+
+// ----- COIN DETAIL
+// The screen a Live prices row opens (same idea as MetaMask's token page):
+// price + 24h change, a price chart with a few time ranges, market stats, a
+// short description, and a Swap button. Swap here means "swap INTO this
+// coin": it is only offered when the coin exists on the selected network as

```

```

+// the native coin or as a token the person has added (matched by symbol) --
+// this wallet never guesses a token contract address. The Swap screen then
+// lets them choose which of their own holdings to pay with.
+let coinDetail = { coin: null, days: 7, gen: 0, chartGen: 0, chartPoints: [], sw
+
+function normalizeCoinSymbol(s) {
+  const up = String(s || "").toUpperCase();
+  return up === "MATIC" ? "POL" : up; // Polygon's native coin was renamed
+}
+
+// Native coin + tracked tokens on the selected network, with balances.
+async function getHeldAssets() {
+  const assets = [];
+  if (!currentStatus || !currentStatus.selectedAddress || !currentNetwork) retur
+  try {
+    const bal = await sendMsg("TM_GET_BALANCE", { address: currentStatus.selecte
+    assets.push({
+      key: "native", address: "", symbol: bal.symbol || currentNetwork.nativeCur
+      decimals: bal.decimals, balance: ethers.utils.formatUnits(bal.balanceWei,
+    });
+  } catch (e) {
+    assets.push({ key: "native", address: "", symbol: currentNetwork.nativeCurre
+  }
+  try {
+    const res = await sendMsg("TM_GET_TRACKED_TOKEN_BALANCES");
+    (res.tokens || []).forEach((t) => {
+      if (t.error) return;
+      assets.push({
+        key: t.address, address: t.address, symbol: t.symbol, decimals: t.decima
+        balance: ethers.utils.formatUnits(t.balanceWei, t.decimals),
+      });
+    });
+  } catch (e) { /* best-effort: native coin only */ }
+  return assets;
+}
+
+function assetLabel(a) {
+  const n = Number(a.balance);
+  return `${a.symbol} (${isFinite(n) ? n.toLocaleString(undefined, { maximumFrac
+}
+
+// ---- Swap screen asset pickers (From = what you hold, To = any of your assets
+let swapPopulateGen = 0;
+
+function syncSwapAsset(side) {
+  const sel = `${swap}-${side}-select`;
+  const input = `${swap}-${side}-custom`;

```

```

+   if (sel.value === "custom") {
+     input.classList.remove("hidden");
+     input.value = "";
+   } else {
+     input.classList.add("hidden");
+     input.value = sel.value === "native" ? "" : sel.value; // blank = native coi
+   }
+   $("swap-quote-display").classList.add("hidden"); // a changed pair invalidates
+}
+
+async function populateSwapSelects(pre) {
+  const myGen = ++swapPopulateGen;
+  const held = await getHeldAssets();
+  if (myGen !== swapPopulateGen) return;
+  const fromSel = $("swap-from-select");
+  const toSel = $("swap-to-select");
+  const toKey = pre && pre.toKey;
+  const fill = (sel, items) => {
+    sel.innerHTML = "";
+    items.forEach((a) => {
+      const o = document.createElement("option");
+      o.value = a.key;
+      o.textContent = assetLabel(a);
+      sel.appendChild(o);
+    });
+    const custom = document.createElement("option");
+    custom.value = "custom";
+    custom.textContent = TM_I18N.t("swap.customAddressOption");
+    sel.appendChild(custom);
+  };
+  // "From" only offers what the person actually holds (plus a paste-address
+  // escape hatch); the coin being bought is never offered as its own source.
+  let fromItems = held.filter((a) => Number(a.balance) > 0 && a.key !== toKey);
+  if (!fromItems.length) fromItems = held.filter((a) => a.key === "native" && a.
+  fill(fromSel, fromItems);
+  fill(toSel, held);
+  if (toKey && held.some((a) => a.key === toKey)) toSel.value = toKey;
+  else toSel.value = (held.find((a) => a.key !== fromSel.value) || held[0] || {
+  syncSwapAsset("from");
+  syncSwapAsset("to");
+}
+
+["from", "to"].forEach((side) => {
+  `${`swap-${side}-select`}.addEventListener("change", () => syncSwapAsset(side))
+});
+
+// ---- Coin screen

```

```

+function renderCoinPrice(price, change) {
+  $("coin-price").textContent = price == null ? TM_I18N.t("prices.naText") : TM_
+  const el = $("coin-change");
+  if (typeof change === "number") {
+    el.className = `price-change ${change >= 0 ? "up" : "down"}`;
+    el.textContent = `${change >= 0 ? "+" : ""}${change.toFixed(2)}%`;
+  } else {
+    el.className = "price-change hidden";
+  }
+}
+
+function coinStat(label, value) {
+  const d = document.createElement("div");
+  d.className = "coin-stat";
+  const k = document.createElement("span");
+  k.className = "k";
+  k.textContent = label;
+  const v = document.createElement("span");
+  v.className = "v";
+  v.textContent = value;
+  d.appendChild(k);
+  d.appendChild(v);
+  return d;
+}
+
+async function openCoinDetail(c, fromScreen) {
+  const gen = ++coinDetail.gen;
+  coinDetail.coin = c;
+  coinDetail.swapTarget = null;
+  $("coin-back").dataset.back = fromScreen || "screen-prices";
+  hideError("coin-error");
+  $("coin-icon").innerHTML = tokenIconHtml(c.symbol, c.image);
+  $("coin-name").textContent = c.name;
+  $("coin-symbol").textContent = c.symbol;
+  $("coin-rank").classList.add("hidden");
+  renderCoinPrice(c.price, c.change24h);
+  $("coin-stats").innerHTML = "";
+  $("coin-about").textContent = TM_I18N.t("prices.loading");
+  $("coin-link-cg").href = c.url || "https://www.coingecko.com/";
+  const btn = $("btn-coin-swap");
+  btn.textContent = TM_I18N.t("coin.swapBtn", { symbol: c.symbol });
+  btn.disabled = true;
+  $("coin-holding").classList.add("hidden");
+  $("coin-swap-note").classList.add("hidden");
+  document.querySelectorAll(".coin-range").forEach((b) => b.classList.toggle("ac
+  showScreen("screen-coin");
+  loadCoinChart(gen);

```

```

+ loadCoinInfo(gen);
+ loadCoinSwapState(gen);
+}
+
+async function loadCoinInfo(gen) {
+  const c = coinDetail.coin;
+  try {
+    const d = await TM_PRICES.getCoinDetail(c.symbol, currentCurrency);
+    if (gen !== coinDetail.gen) return;
+    if (d.price !== null) renderCoinPrice(d.price, d.change24h !== null ? d.change
+    if (d.rank !== null) {
+      $("coin-rank").textContent = `#${d.rank}`;
+      $("coin-rank").classList.remove("hidden");
+    }
+    const stats = $("coin-stats");
+    stats.innerHTML = "";
+    const fm = (n) => TM_PRICES.formatMoney(n, currentCurrency, { price: true })
+    if (d.marketCap !== null) stats.appendChild(coinStat(TM_I18N.t("coin.marketCa
+    if (d.volume !== null) stats.appendChild(coinStat(TM_I18N.t("coin.volume24h")
+    if (d.high24h !== null) stats.appendChild(coinStat(TM_I18N.t("coin.high24h"),
+    if (d.low24h !== null) stats.appendChild(coinStat(TM_I18N.t("coin.low24h"), f
+    if (d.circulatingSupply !== null) stats.appendChild(coinStat(TM_I18N.t("coin.
+    if (d.ath !== null) {
+      const pct = d.athChange !== null ? ` (${d.athChange.toFixed(1)}%)` : "";
+      stats.appendChild(coinStat(TM_I18N.t("coin.ath"), fm(d.ath) + pct));
+    }
+    $("coin-about").textContent = d.description || TM_I18N.t("coin.aboutUnavaila
+  } catch (e) {
+    if (gen !== coinDetail.gen) return;
+    $("coin-about").textContent = TM_I18N.t("coin.aboutUnavailable");
+    showError("coin-error", TM_I18N.t("coin.loadError"));
+  }
+}
+
+async function loadCoinChart(gen) {
+  const c = coinDetail.coin;
+  const token = ++coinDetail.chartGen;
+  const box = $("coin-chart");
+  box.className = "coin-chart";
+  box.innerHTML = `<div class="coin-chart-msg">${escapeHtml(TM_I18N.t("prices.lo
+  $("coin-chart-readout").textContent = "";
+  try {
+    const pts = await TM_PRICES.getCoinChart(c.symbol, currentCurrency, coinDeta
+    if (gen !== coinDetail.gen || token !== coinDetail.chartGen) return;
+    drawCoinChart(pts);
+  } catch (e) {
+    if (gen !== coinDetail.gen || token !== coinDetail.chartGen) return;

```

```

+   box.innerHTML = `<div class="coin-chart-msg">${escapeHtml(TM_I18N.t("coin.ch
+   }
+}
+
+function coinRangeSummary() {
+  const pts = coinDetail.chartPoints;
+  if (pts.length < 2) return "";
+  const first = pts[0][1];
+  const last = pts[pts.length - 1][1];
+  const pct = first ? ((last - first) / first) * 100 : 0;
+  const label = { 1: "24H", 7: "7D", 30: "1M", 365: "1Y" }[coinDetail.days] || "
+  return `${label}: ${pct >= 0 ? "+" : ""}${pct.toFixed(2)}%`;
+}
+
+function drawCoinChart(pts) {
+  const box = $("coin-chart");
+  coinDetail.chartPoints = pts;
+  if (!pts || pts.length < 2) {
+    box.innerHTML = `<div class="coin-chart-msg">${escapeHtml(TM_I18N.t("coin.ch
+    return;
+  }
+  const W = 320, H = 130, PAD = 6;
+  const prices = pts.map((p) => p[1]);
+  const min = Math.min(...prices);
+  const max = Math.max(...prices);
+  const span = max - min || 1;
+  const xs = pts.map((_, i) => (i / (pts.length - 1)) * W);
+  const ys = prices.map((p) => H - PAD - ((p - min) / span) * (H - PAD * 2));
+  const line = xs.map((x, i) => `${i ? "L" : "M"}${x.toFixed(1)} ${ys[i].toFixed
+  const area = `${line} L${W} ${H} L0 ${H} Z`;
+  box.className = `coin-chart ${prices[prices.length - 1] >= prices[0] ? "up" :
+  box.innerHTML = `<svg viewBox="0 0 ${W} ${H}" preserveAspectRatio="none" role=
+    <path d="${area}" fill="currentColor" fill-opacity="0.12" stroke="none"></pa
+    <path d="${line}" fill="none" stroke="currentColor" stroke-width="2" stroke=
+    <line id="coin-chart-cursor" x1="0" y1="0" x2="0" y2="${H}" stroke="currentC
+  </svg>`;
+  $("coin-chart-readout").textContent = coinRangeSummary();
+}
+
+// Drag/hover over the chart to read the price at that moment.
+(function wireCoinChartScrub() {
+  const box = $("coin-chart");
+  const show = (e) => {
+    const pts = coinDetail.chartPoints;
+    if (!pts || pts.length < 2) return;
+    const rect = box.getBoundingClientRect();
+    if (!rect.width) return;

```

```

+   const ratio = Math.min(1, Math.max(0, (e.clientX - rect.left) / rect.width))
+   const idx = Math.round(ratio * (pts.length - 1));
+   const [ts, price] = pts[idx];
+   const cursor = $("coin-chart-cursor");
+   if (cursor) {
+     const x = (idx / (pts.length - 1)) * 320;
+     cursor.setAttribute("x1", x);
+     cursor.setAttribute("x2", x);
+     cursor.setAttribute("opacity", "0.7");
+   }
+   const when = new Date(ts).toLocaleString(undefined, coinDetail.days === 1
+     ? { hour: "numeric", minute: "2-digit" }
+     : coinDetail.days <= 7 ? { month: "short", day: "numeric", hour: "numeric"
+   $("coin-chart-readout").textContent = `${TM_PRICES.formatMoney(price, curren
+   });
+   const reset = () => {
+     const cursor = $("coin-chart-cursor");
+     if (cursor) cursor.setAttribute("opacity", "0");
+     $("coin-chart-readout").textContent = coinRangeSummary();
+   };
+   box.addEventListener("pointermove", show);
+   box.addEventListener("pointerdown", show);
+   box.addEventListener("pointerleave", reset);
+   box.addEventListener("pointerup", reset);
+   box.addEventListener("pointercancel", reset);
+ }) ();
+
+document.querySelectorAll(".coin-range").forEach((b) => {
+  b.addEventListener("click", () => {
+    coinDetail.days = Number(b.dataset.days);
+    document.querySelectorAll(".coin-range").forEach((x) => x.classList.toggle("
+    loadCoinChart(coinDetail.gen);
+  });
+});
+
+async function loadCoinSwapState(gen) {
+  const c = coinDetail.coin;
+  const btn = $("btn-coin-swap");
+  const note = $("coin-swap-note");
+  const showNote = (text) => { note.textContent = text; note.classList.remove("h
+  if (!currentNetwork || !currentStatus || !currentStatus.selectedAddress) {
+    showNote(TM_I18N.t("coin.swapNeedsWallet"));
+    return;
+  }
+  if (!currentNetwork.swapRouter) {
+    showNote(TM_I18N.t("coin.swapNoRouter"));
+    return;

```

```

+   }
+   const held = await getHeldAssets();
+   if (gen !== coinDetail.gen) return;
+   const want = normalizeCoinSymbol(c.symbol);
+   const match = held.find((a) => normalizeCoinSymbol(a.symbol) === want);
+   if (!match) {
+     showNote(TM_I18N.t("coin.swapUnavailableNetwork", { symbol: c.symbol, network: c.network }));
+     return;
+   }
+   coinDetail.swapTarget = match;
+   if (Number(match.balance) > 0) {
+     const holding = $("coin-holding");
+     holding.textContent = TM_I18N.t("coin.holding", {
+       amount: Number(match.balance).toLocaleString(undefined, { maximumFractionDigits: 8 }),
+       symbol: match.symbol,
+     });
+     holding.classList.remove("hidden");
+   }
+   btn.disabled = false;
+   showNote(TM_I18N.t("coin.swapHint"));
+}
+
+$( "btn-coin-swap" ).addEventListener("click", () => {
+  const m = coinDetail.swapTarget;
+  if (!m) return;
+  setupSwapScreen({ toKey: m.key });
+  showScreen("screen-swap");
+});
+
+// ----- ACCOUNT (optional)
+// Username sign-in for the website only. It syncs three display settings
+// (currency, language, watchlist) and nothing else -- never keys, phrases,
+// addresses or balances. See accounts-api.js for the server side. The
+// browser never sends the raw password: it derives an auth key from
+// (username, password) and sends that instead. The whole feature stays
+// hidden unless the server reports that accounts are switched on.
+let accountEnabled = false;
+let accountUser = null;
+let accountMode = "signin";
+let accountApplying = false; // true while applying pulled settings, so that does not
+let accountPushTimer = null;
+
+async function accountApi(method, path, body) {
+  const res = await fetch(path, {
+    method,
+    credentials: "same-origin",
+    headers: Object.assign({ "X-TM-Requested-With": "web" }, body ? { "Content-Type": "application/json" } : {}),
+  });
+  if (res.status === 200) {
+    const json = await res.json();
+    if (method === "POST" || method === "PUT") {
+      if (json.error) {
+        showNote(TM_I18N.t(json.error));
+        return;
+      }
+      if (json.user) {
+        accountUser = json.user;
+      }
+      if (json.mode) {
+        accountMode = json.mode;
+      }
+      if (json.enabled) {
+        accountEnabled = true;
+      }
+    }
+  }
+}

```

```

+   body: body ? JSON.stringify(body) : undefined,
+ });
+ let json = null;
+ try { json = await res.json(); } catch (e) { /* non-JSON reply */ }
+ if (!res.ok) {
+   const err = new Error((json && json.error) || "server_error");
+   err.code = err.message;
+   err.status = res.status;
+   throw err;
+ }
+ return json;
+}
+
+async function deriveAccountAuthKey(username, password) {
+  const enc = new TextEncoder();
+  const material = await crypto.subtle.importKey("raw", enc.encode(password.normalize()), { name: "PBKDF2", hash: "SHA-256", salt: enc.encode("tm-account-v1:" + username) }, true, ["deriveBits"]);
+  const bits = await crypto.subtle.deriveBits(
+    { name: "PBKDF2", hash: "SHA-256", salt: enc.encode("tm-account-v1:" + username) },
+    material,
+    256
+  );
+  return Array.from(new Uint8Array(bits)).map((b) => b.toString(16).padStart(2, "0"));
+}
+
+function accountErrorText(code) {
+  const map = {
+    bad_credentials: "account.errBadLogin",
+    username_taken: "account.errTaken",
+    rate_limited: "account.errRateLimited",
+    unavailable: "account.errUnavailable",
+    bad_request: "account.errUsername",
+  };
+  return TM_I18N.t(map[code] || "account.errGeneric");
+}
+
+function updateAccountUi() {
+  const entry = $("#btn-goto-account");
+  const onboardingLink = $("#btn-onboarding-signin");
+  entry.classList.toggle("hidden", !accountEnabled);
+  onboardingLink.classList.toggle("hidden", !accountEnabled || !!accountUser);
+  if (accountUser) {
+    entry.removeAttribute("data-il8n"); // otherwise a language switch would override
+    entry.textContent = TM_I18N.t("account.signedInAs", { username: accountUser });
+  } else {
+    entry.setAttribute("data-il8n", "account.entryBtn");
+    entry.textContent = TM_I18N.t("account.entryBtn");
+  }
+}

```

```

+ $("account-signed-out").classList.toggle("hidden", !!accountUser);
+ $("account-signed-in").classList.toggle("hidden", !accountUser);
+ $("account-signed-in-as").textContent = accountUser ? TM_I18N.t("account.signe
+ $("account-back").dataset.back = $("account-back").dataset.back || "screen-set
+}
+
+function setAccountMode(mode) {
+  accountMode = mode;
+  document.querySelectorAll(".account-tab").forEach((b) => b.classList.toggle("a
+  const creating = mode === "create";
+  $("account-confirm-row").classList.toggle("hidden", !creating);
+  $("account-username-hint").classList.toggle("hidden", !creating);
+  $("account-password").setAttribute("autocomplete", creating ? "new-password" :
+  $("btn-account-submit").textContent = TM_I18N.t(creating ? "account.createSubm
+  $("btn-account-submit").setAttribute("data-i18n", creating ? "account.createSu
+  hideError("account-error");
+}
+
+function openAccountScreen(fromScreen) {
+  $("account-back").dataset.back = fromScreen;
+  hideError("account-error");
+  $("account-status").classList.add("hidden");
+  $("account-password").value = "";
+  $("account-password2").value = "";
+  $("account-delete-password").value = "";
+  updateAccountUi();
+  showScreen("screen-account");
+}
+
+function accountSnapshot() {
+  return { currency: currentCurrency, language: TM_I18N.getLanguage(), watchlist
+}
+
+// Pull: the server's values win for currency and language; watchlists are
+// merged so stars set before signing in aren't lost. Returns true if the
+// merged result differs from what the server had (so we push it back).
+async function applyRemoteSettings(remote) {
+  if (!remote) return true; // nothing stored yet -- push what this device has
+  accountApplying = true;
+  let serverMissedSomething = false;
+  try {
+    if (remote.currency && TM_PRICES.SUPPORTED_CURRENCIES[remote.currency] && re
+    currentCurrency = remote.currency;
+    chrome.storage.local.set({ [TM_CURRENCY_KEY]: currentCurrency });
+    const sel = $("currency-select-settings");
+    if (sel) sel.value = currentCurrency;
+  }

```

```

+   if (remote.language && remote.language !== TM_I18N.getLanguage()) {
+       TM_I18N.setLanguage(remote.language);
+       document.querySelectorAll(".language-select").forEach((sel) => { sel.value
+   })
+   const remoteList = Array.isArray(remote.watchlist) ? remote.watchlist : [];
+   const before = watchlistSymbols.size;
+   remoteList.forEach((sym) => watchlistSymbols.add(String(sym).toUpperCase()))
+   if (watchlistSymbols.size !== before) {
+       chrome.storage.local.set({ [TM_WATCHLIST_KEY]: Array.from(watchlistSymbols
+   })
+   serverMissedSomething = Array.from(watchlistSymbols).some((s) => !remoteList
+   // Re-render anything that shows prices or translated text.
+   try {
+       if (!$("screen-prices").classList.contains("hidden")) refreshPrices();
+       refreshMainPricesCard();
+       if (currentStatus && currentStatus.unlocked) { refreshBalanceUsd(); refres
+   } catch (e) { /* best effort */ }
+   } finally {
+       accountApplying = false;
+   }
+   return serverMissedSomething;
+}
+
+function scheduleAccountPush() {
+   if (!accountUser || accountApplying) return;
+   clearTimeout(accountPushTimer);
+   accountPushTimer = setTimeout(pushAccountSettings, 1500);
+}
+
+async function pushAccountSettings() {
+   if (!accountUser) return;
+   try {
+       await accountApi("PUT", "/api/account/settings", { settings: accountSnapshot
+   } catch (e) {
+       if (e.status === 401) { accountUser = null; updateAccountUi(); } // session
+   }
+}
+
+async function onAccountSignedIn(resp) {
+   accountUser = resp.username;
+   const needsPush = await applyRemoteSettings(resp.settings);
+   if (needsPush) await pushAccountSettings();
+   updateAccountUi();
+}
+
+$("#btn-goto-account").addEventListener("click", () => openAccountScreen("screen-
+$("#btn-onboarding-signin").addEventListener("click", () => openAccountScreen("sc

```

```

+document.querySelectorAll(".account-tab").forEach((b) => b.addEventListener("cli
+
+$("#btn-account-submit").addEventListener("click", async () => {
+  hideError("account-error");
+  $(".account-status").classList.add("hidden");
+  const username = $(".account-username").value.trim().toLowerCase();
+  const password = $(".account-password").value;
+  if (!/^[a-z0-9_]{3,24}$/i.test(username)) return showError("account-error", TM_
+  if (accountMode === "create") {
+    if (password.length < 10) return showError("account-error", TM_I18N.t("accou
+    if (password !== $(".account-password2").value) return showError("account-err
+  } else if (!password) {
+    return showError("account-error", TM_I18N.t("account.errBadLogin"));
+  }
+  const btn = $(".btn-account-submit");
+  btn.disabled = true;
+  $(".account-status").textContent = TM_I18N.t("account.working");
+  $(".account-status").classList.remove("hidden");
+  try {
+    const authKey = await deriveAccountAuthKey(username, password);
+    const resp = await accountApi("POST", accountMode === "create" ? "/api/accou
+    $(".account-password").value = "";
+    $(".account-password2").value = "";
+    await onAccountSignedIn(resp);
+    $(".account-status").textContent = TM_I18N.t("account.synced");
+  } catch (e) {
+    $(".account-status").classList.add("hidden");
+    showError("account-error", accountErrorText(e.code));
+  } finally {
+    btn.disabled = false;
+  }
+});
+
+$("#btn-account-signout").addEventListener("click", async () => {
+  try { await accountApi("POST", "/api/account/logout", {}); } catch (e) { /* si
+  accountUser = null;
+  clearTimeout(accountPushTimer);
+  updateAccountUi();
+  setAccountMode("signin");
+});
+
+$("#btn-account-delete").addEventListener("click", async () => {
+  hideError("account-error");
+  const password = $(".account-delete-password").value;
+  if (!password) return showError("account-error", TM_I18N.t("account.deleteNeed
+  if (!window.confirm(TM_I18N.t("account.deleteConfirm"))) return;
+  try {

```

```

+   const authKey = await deriveAccountAuthKey(accountUser, password);
+   await accountApi("POST", "/api/account/delete", { authKey });
+   accountUser = null;
+   clearTimeout(accountPushTimer);
+   $("account-delete-password").value = "";
+   updateAccountUi();
+   setAccountMode("signin");
+ } catch (e) {
+   showError("account-error", accountErrorText(e.code));
+ }
+});
+
+// Keep the server copy in step whenever one of the synced settings changes.
+document.addEventListener("tm-language-changed", () => { updateAccountUi(); sche
+
+async function initAccount() {
+  try {
+    const res = await fetch("/api/account/status", { credentials: "same-origin"
+    const st = await res.json();
+    accountEnabled = !!st && st.enabled;
+  } catch (e) {
+    accountEnabled = false; // no API here (older server, file://, offline) -- f
+  }
+  updateAccountUi();
+  if (!accountEnabled) return;
+  try {
+    const me = await accountApi("GET", "/api/account/me");
+    accountUser = me.username;
+    const needsPush = await applyRemoteSettings(me.settings);
+    if (needsPush) scheduleAccountPush();
+  } catch (e) {
+    accountUser = null;
+  }
+  updateAccountUi();
+}
+
+// Compact live-prices card on the main screen -- just the first handful of
+// PRICE_BOARD's coins, at a glance, without navigating away. Best-effort
+// like the balance/token refreshes above: a rate-limited or unreachable
+@@ -1775,13 +2392,14 @@
+});
+
+// ----- SWAP
+function setupSwapScreen() {
+function setupSwapScreen(pre) {
+  hideError("swap-error");
+  $("swap-status").classList.add("hidden");

```

```

    $("swap-quote-display").classList.add("hidden");
    const unsupported = !currentNetwork.swapRouter;
    $("swap-unsupported").classList.toggle("hidden", !unsupported);
    $("swap-form").classList.toggle("hidden", unsupported);
+   if (!unsupported) populateSwapSelects(pre || {}).catch(() => {});
  }

  $("btn-swap-quote").addEventListener("click", async () => {
@@ -2152,16 +2770,23 @@
    function populateCurrencySelect() {
      const sel = $("currency-select-settings");
      sel.innerHTML = "";
+     const groups = { fiat: document.createElement("optgroup"), crypto: document.cr
+     groups.fiat.label = "Currencies";
+     groups.crypto.label = "Crypto";
      Object.keys(TM_PRICES.SUPPORTED_CURRENCIES).forEach((code) => {
+       const info = TM_PRICES.SUPPORTED_CURRENCIES[code];
        const opt = document.createElement("option");
        opt.value = code;
-       opt.textContent = `${TM_PRICES.SUPPORTED_CURRENCIES[code].symbol} ${TM_PRICE
-       sel.appendChild(opt);
+       opt.textContent = `${info.label} - ${info.name} (${info.symbol})`;
+       (groups[info.type] || groups.fiat).appendChild(opt);
      });
+     sel.appendChild(groups.fiat);
+     sel.appendChild(groups.crypto);
      sel.value = currentCurrency;
      sel.addEventListener("change", async (e) => {
        currentCurrency = e.target.value;
        chrome.storage.local.set({ [TM_CURRENCY_KEY]: currentCurrency });
+       scheduleAccountPush();
        // Re-render whatever's currently showing a price so the switch feels
        // immediate rather than waiting for the next natural refresh.
        if (currentStatus && currentStatus.unlocked) {
@@ -2673,6 +3298,7 @@
        await loadWatchlist();
        await loadCurrency();
        populateCurrencySelect();
+       initAccount(); // not awaited: the wallet must never wait on the network
        await loadSendDelay();
        populateSendDelaySelect();
        await loadAutoLockMinutes();
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/ar.js
--- /tmp/orig/toKen-exchange-main/ar.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/ar.js 2026-09-24 23:22:06.588022498 +0000
@@ -73,6 +73,61 @@
    "prices.description": "CoinGecko أسعار الدولار الأمريكي مأخوذة من واجهة"

```

```
"prices.loading": "جارٍ التحميل...",
"prices.naText": "غير متاح",
+ "prices.tabCrypto": "العملات الرقمية",
+ "prices.tabCurrencies": "العملات",
+ "prices.searchPlaceholder": "بحث...",
+ "prices.currenciesNote": "يُمَِّمة وحدة واحدة من كل عملة بعملة العرض التي اخترتها",
+ "prices.noResults": "لا توجد نتائج",
+ "prices.viewCoin": "عرض {name}",
+ "coin.swapBtn": "{symbol} مبادلة إلى",
+ "coin.swapHint": "ستختار من الشاشة التالية أيًا من أصولك تريد الدفع به",
+ "coin.swapNeedsWallet": "افتح محفظتك للمبادلة",
+ "coin.swapNoRouter": "المبادلة غير مفعّلة على هذه الشبكة بعد",
+ "coin.swapUnavailableNetwork": "لافة رمز {network} على {symbol} لا يمكن مبادلة",
+ "coin.holding": "{amount} {symbol} لديك",
+ "coin.statsTitle": "إحصاءات السوق",
+ "coin.aboutTitle": "نبذة",
+ "coin.aboutUnavailable": "لا يتوفر وصف حاليًا",
+ "coin.marketCap": "القيمة السوقية",
+ "coin.volume24h": "حجم 24 ساعة",
+ "coin.high24h": "أعلى سعر خلال 24 ساعة",
+ "coin.low24h": "أدنى سعر خلال 24 ساعة",
+ "coin.supply": "المعروض المتداول",
+ "coin.ath": "أعلى سعر على الإطلاق",
+ "coin.viewCoinGecko": "CoinGecko عرض على",
+ "coin.loadError": "تعدّر تحميل المزيد من التفاصيل عن هذه العملة الآن",
+ "coin.chartUnavailable": "الرسم البياني غير متاح حاليًا",
+ "account.title": "تسجيل الدخول",
+ "account.intro": "ين أجهزتك. لا يخزّن أبدًا محفظتك أو مفاتيحك أو عبارة الاسترداد",
+ "account.tabSignIn": "تسجيل الدخول",
+ "account.tabCreate": "إنشاء حساب",
+ "account.usernameLabel": "اسم المستخدم",
+ "account.usernameHint": "حرفًا: أحرف أو أرقام أو شرطة سفلية 3-24",
+ "account.passwordLabel": "كلمة المرور",
+ "account.confirmLabel": "تأكيد كلمة المرور",
+ "account.walletWarning": "في مكان آمن -- فهي الطريقة الوحيدة لاستعادة أموالك",
+ "account.signInSubmit": "تسجيل الدخول",
+ "account.createSubmit": "إنشاء حساب",
+ "account.working": "جارٍ التنفيذ",
+ "account.synced": "تمت المزامنة",
+ "account.signedInAs": "{username} تم تسجيل الدخول باسم",
+ "account.syncNote": "المزامنة: عملة العرض واللغة وقائمة المتابعة",
+ "account.signOut": "تسجيل الخروج",
+ "account.deleteLabel": "حذف الحساب",
+ "account.deletePlaceholder": "كلمة المرور للتأكيد",
+ "account.deleteBtn": "حذف الحساب",
+ "account.deleteNeedsPassword": "أدخل كلمة المرور لحذف حسابك",
+ "account.deleteConfirm": "امنة. لن يتأثر محفظتك أو أموالك. هل تريد المتابعة؟"
```

```

+   "account.entryBtn": "تسجيل الدخول / إنشاء حساب",
+   "account.errUsername": "مستخدم من 3 إلى 24 حرفًا: أحرف أو أرقام أو شرطة سفلية",
+   "account.errPasswordShort": ".يجب ألا تقل كلمة المرور عن 10 أحرف",
+   "account.errMismatch": ".كلمتا المرور غير متطابقتين",
+   "account.errBadLogin": ".اسم المستخدم أو كلمة المرور غير صحيحة",
+   "account.errTaken": ".اسم المستخدم هذا مُستخدم بالفعل",
+   "account.errRateLimited": "محاولات كثيرة جدًا. يرجى الانتظار والمحاولة مرة أخرى",
+   "account.errUnavailable": ".الحسابات غير متاحة حاليًا",
+   "account.errGeneric": ".حدث خطأ ما. يرجى المحاولة مرة أخرى",
+   "swap.customAddressOption": ".رمز آخر (الصق العنوان)",

```

```

    "buy.title": "شراء عملات رقمية",
    "buy.description1": "خارجية منفصلة -- لا تطلع، MoonPay خدمة الشراء مقدّمة عبر"
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/en.js
--- /tmp/orig/toKen-exchange-main/en.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/en.js 2026-09-24 23:22:06.587206683 +0000
@@ -99,6 +99,61 @@
    "prices.naText": "n/a",
    "prices.watchlistToggle": "Star to pin to top",
    "prices.cardUnavailable": "Prices unavailable right now.",
+   "prices.tabCrypto": "Crypto",
+   "prices.tabCurrencies": "Currencies",
+   "prices.searchPlaceholder": "Search...",
+   "prices.currenciesNote": "What 1 unit of each currency is worth in your disp
+   "prices.noResults": "No results.",
+   "prices.viewCoin": "View {name}",
+   "coin.swapBtn": "Swap for {symbol}",
+   "coin.swapHint": "You'll choose which of your holdings to pay with on the ne
+   "coin.swapNeedsWallet": "Unlock your wallet to swap.",
+   "coin.swapNoRouter": "Swapping isn't enabled on this network yet.",
+   "coin.swapUnavailableNetwork": "{symbol} can't be swapped on {network} yet.
+   "coin.holding": "You hold {amount} {symbol}",
+   "coin.statsTitle": "Market stats",
+   "coin.aboutTitle": "About",
+   "coin.aboutUnavailable": "No description available right now.",
+   "coin.marketCap": "Market cap",
+   "coin.volume24h": "24h volume",
+   "coin.high24h": "24h high",
+   "coin.low24h": "24h low",
+   "coin.supply": "Circulating supply",
+   "coin.ath": "All-time high",
+   "coin.viewCoinGecko": "View on CoinGecko",
+   "coin.loadError": "Couldn't load more details for this coin right now.",
+   "coin.chartUnavailable": "Chart unavailable right now.",
+   "account.title": "Sign in",
+   "account.intro": "Optional. An account syncs your display currency, language
+   "account.tabSignIn": "Sign in",

```

```

+   "account.tabCreate": "Create account",
+   "account.usernameLabel": "Username",
+   "account.usernameHint": "3-24 characters: letters, numbers or underscore.",
+   "account.passwordLabel": "Password",
+   "account.confirmLabel": "Confirm password",
+   "account.walletWarning": "This password can't recover your wallet. Keep your",
+   "account.signInSubmit": "Sign in",
+   "account.createSubmit": "Create account",
+   "account.working": "Working...",
+   "account.synced": "Synced.",
+   "account.signedInAs": "Signed in as {username}",
+   "account.syncNote": "Syncing: display currency, language and watchlist.",
+   "account.signOut": "Sign out",
+   "account.deleteLabel": "Delete account",
+   "account.deletePlaceholder": "Password to confirm",
+   "account.deleteBtn": "Delete account",
+   "account.deleteNeedsPassword": "Enter your password to delete your account."
+   "account.deleteConfirm": "This deletes your account and its synced settings.
+   "account.entryBtn": "Sign in / Create account",
+   "account.errUsername": "Username must be 3-24 characters: letters, numbers o
+   "account.errPasswordShort": "Password must be at least 10 characters.",
+   "account.errMismatch": "Passwords don't match.",
+   "account.errBadLogin": "Incorrect username or password.",
+   "account.errTaken": "That username is taken.",
+   "account.errRateLimited": "Too many attempts. Please wait and try again.",
+   "account.errUnavailable": "Accounts aren't available right now.",
+   "account.errGeneric": "Something went wrong. Please try again.",
+   "swap.customAddressOption": "Other token (paste address)...",

    "predictions.title": "Trending Markets",
    "predictions.description": "Public odds from Polymarket's prediction markets
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/es.js
--- /tmp/orig/toKen-exchange-main/es.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/es.js 2026-09-24 23:22:06.588746854 +0000
@@ -73,6 +73,61 @@
    "prices.description": "Precios en USD de la API pública de CoinGecko. Solo c
    "prices.loading": "Cargando...",
    "prices.naText": "n/d",
+   "prices.tabCrypto": "Cripto",
+   "prices.tabCurrencies": "Divisas",
+   "prices.searchPlaceholder": "Buscar...",
+   "prices.currenciesNote": "Lo que vale 1 unidad de cada divisa en tu moneda d
+   "prices.noResults": "Sin resultados.",
+   "prices.viewCoin": "Ver {name}",
+   "coin.swapBtn": "Cambiar a {symbol}",
+   "coin.swapHint": "En la siguiente pantalla elegirás con cuál de tus activos
+   "coin.swapNeedsWallet": "Desbloquea tu billetera para cambiar.",

```

```
+ "coin.swapNoRouter": "El intercambio aún no está habilitado en esta red.",
+ "coin.swapUnavailableNetwork": "Todavía no se puede cambiar {symbol} en {net
+ "coin.holding": "Tienes {amount} {symbol}",
+ "coin.statsTitle": "Datos de mercado",
+ "coin.aboutTitle": "Acerca de",
+ "coin.aboutUnavailable": "No hay descripción disponible por ahora.",
+ "coin.marketCap": "Capitalización",
+ "coin.volume24h": "Volumen 24 h",
+ "coin.high24h": "Máximo 24 h",
+ "coin.low24h": "Mínimo 24 h",
+ "coin.supply": "Suministro circulante",
+ "coin.ath": "Máximo histórico",
+ "coin.viewCoinGecko": "Ver en CoinGecko",
+ "coin.loadError": "No se pudieron cargar más detalles de esta moneda ahora."
+ "coin.chartUnavailable": "Gráfico no disponible por ahora.",
+ "account.title": "Iniciar sesión",
+ "account.intro": "Opcional. Una cuenta sincroniza tu moneda de visualización
+ "account.tabSignIn": "Iniciar sesión",
+ "account.tabCreate": "Crear cuenta",
+ "account.usernameLabel": "Nombre de usuario",
+ "account.usernameHint": "3-24 caracteres: letras, números o guion bajo.",
+ "account.passwordLabel": "Contraseña",
+ "account.confirmLabel": "Confirmar contraseña",
+ "account.walletWarning": "Esta contraseña no puede recuperar tu billetera. G
+ "account.signInSubmit": "Iniciar sesión",
+ "account.createSubmit": "Crear cuenta",
+ "account.working": "Procesando...",
+ "account.synced": "Sincronizado.",
+ "account.signedInAs": "Sesión iniciada como {username}",
+ "account.syncNote": "Sincronizando: moneda de visualización, idioma y lista
+ "account.signOut": "Cerrar sesión",
+ "account.deleteLabel": "Eliminar cuenta",
+ "account.deletePlaceholder": "Contraseña para confirmar",
+ "account.deleteBtn": "Eliminar cuenta",
+ "account.deleteNeedsPassword": "Ingresa tu contraseña para eliminar tu cuent
+ "account.deleteConfirm": "Esto elimina tu cuenta y su configuración sincróni
+ "account.entryBtn": "Iniciar sesión / Crear cuenta",
+ "account.errUsername": "El nombre de usuario debe tener 3-24 caracteres: let
+ "account.errPasswordShort": "La contraseña debe tener al menos 10 caracteres
+ "account.errMismatch": "Las contraseñas no coinciden.",
+ "account.errBadLogin": "Usuario o contraseña incorrectos.",
+ "account.errTaken": "Ese nombre de usuario ya está en uso.",
+ "account.errRateLimited": "Demasiados intentos. Espera y vuelve a intentarlo
+ "account.errUnavailable": "Las cuentas no están disponibles en este momento.
+ "account.errGeneric": "Algo salió mal. Inténtalo de nuevo.",
+ "swap.customAddressOption": "Otro token (pega la dirección)...",
```

```

    "buy.title": "Comprar criptomonedas",
    "buy.description1": "La función Comprar funciona con MoonPay, un servicio de
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/fr.js
--- /tmp/orig/toKen-exchange-main/fr.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/fr.js 2026-09-24 23:22:06.589138944 +0000
@@ -74,6 +74,61 @@
    "prices.description": "Cours en USD fournis par l'API publique de CoinGecko.
    "prices.loading": "Chargement...",
    "prices.naText": "n/d",
+   "prices.tabCrypto": "Crypto",
+   "prices.tabCurrencies": "Devises",
+   "prices.searchPlaceholder": "Rechercher...",
+   "prices.currenciesNote": "Ce que vaut 1 unité de chaque devise dans votre de
+   "prices.noResults": "Aucun résultat.",
+   "prices.viewCoin": "Voir {name}",
+   "coin.swapBtn": "Échanger contre {symbol}",
+   "coin.swapHint": "Vous choisirez avec quel actif payer à l'écran suivant.",
+   "coin.swapNeedsWallet": "Déverrouillez votre portefeuille pour échanger.",
+   "coin.swapNoRouter": "L'échange n'est pas encore activé sur ce réseau.",
+   "coin.swapUnavailableNetwork": "{symbol} ne peut pas encore être échangé sur
+   "coin.holding": "Vous détenez {amount} {symbol}",
+   "coin.statsTitle": "Données de marché",
+   "coin.aboutTitle": "À propos",
+   "coin.aboutUnavailable": "Aucune description disponible pour le moment.",
+   "coin.marketCap": "Capitalisation",
+   "coin.volume24h": "Volume 24 h",
+   "coin.high24h": "Plus haut 24 h",
+   "coin.low24h": "Plus bas 24 h",
+   "coin.supply": "Offre en circulation",
+   "coin.ath": "Plus haut historique",
+   "coin.viewCoinGecko": "Voir sur CoinGecko",
+   "coin.loadError": "Impossible de charger plus de détails sur cette crypto po
+   "coin.chartUnavailable": "Graphique indisponible pour le moment.",
+   "account.title": "Se connecter",
+   "account.intro": "Facultatif. Un compte synchronise votre devise d'affichage
+   "account.tabSignIn": "Se connecter",
+   "account.tabCreate": "Créer un compte",
+   "account.usernameLabel": "Nom d'utilisateur",
+   "account.usernameHint": "3 à 24 caractères : lettres, chiffres ou tiret bas.
+   "account.passwordLabel": "Mot de passe",
+   "account.confirmLabel": "Confirmer le mot de passe",
+   "account.walletWarning": "Ce mot de passe ne peut pas récupérer votre portef
+   "account.signInSubmit": "Se connecter",
+   "account.createSubmit": "Créer un compte",
+   "account.working": "Chargement...",
+   "account.synced": "Synchronisé.",
+   "account.signedInAs": "Connecté en tant que {username}",

```

```

+   "account.syncNote": "Synchronisation : devise d'affichage, langue et liste d
+   "account.signOut": "Se déconnecter",
+   "account.deleteLabel": "Supprimer le compte",
+   "account.deletePlaceholder": "Mot de passe pour confirmer",
+   "account.deleteBtn": "Supprimer le compte",
+   "account.deleteNeedsPassword": "Saisissez votre mot de passe pour supprimer
+   "account.deleteConfirm": "Cela supprime votre compte et ses paramètres synch
+   "account.entryBtn": "Se connecter / Créer un compte",
+   "account.errUsername": "Le nom d'utilisateur doit comporter 3 à 24 caractère
+   "account.errPasswordShort": "Le mot de passe doit comporter au moins 10 cara
+   "account.errMismatch": "Les mots de passe ne correspondent pas.",
+   "account.errBadLogin": "Nom d'utilisateur ou mot de passe incorrect.",
+   "account.errTaken": "Ce nom d'utilisateur est déjà pris.",
+   "account.errRateLimited": "Trop de tentatives. Veuillez patienter puis réess
+   "account.errUnavailable": "Les comptes ne sont pas disponibles pour le momen
+   "account.errGeneric": "Une erreur s'est produite. Veuillez réessayer.",
+   "swap.customAddressOption": "Autre jeton (coller l'adresse)...",

    "buy.title": "Acheter des cryptos",
    "buy.description1": "L'achat est propulsé par MoonPay, un service tiers indé

diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/hi.js
--- /tmp/orig/toKen-exchange-main/hi.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/hi.js 2026-09-24 23:22:06.590117111 +0000
@@ -74,6 +74,61 @@
    "prices.description": "CoinGecko के पब्लिक API से USD में कीमतें। यह पूरी तरह जानकारी के
    "prices.loading": "लोड हो रहा है...",
    "prices.noText": "उपलब्ध नहीं",
+   "prices.tabCrypto": "क्रिप्टो",
+   "prices.tabCurrencies": "मुद्राएँ",
+   "prices.searchPlaceholder": "खोजें...",
+   "prices.currenciesNote": "आपकी चुनी हुई डिस्प्ले मुद्रा में हर मुद्रा की 1 इकाई का मूल्य।",
+   "prices.noResults": "कोई परिणाम नहीं।",
+   "prices.viewCoin": "{name} देखें",
+   "coin.swapBtn": "{symbol} में स्वेप करें",
+   "coin.swapHint": "अगली स्क्रीन पर आप चुनेंगे कि अपनी किस होल्डिंग से भुगतान करना है।",
+   "coin.swapNeedsWallet": "स्वेप करने के लिए अपना वॉलेट अनलॉक करें।",
+   "coin.swapNoRouter": "इस नेटवर्क पर स्वेप अभी चालू नहीं है।",
+   "coin.swapUnavailableNetwork": "{network} पर {symbol} अभी स्वेप नहीं हो सकता। यह ने
+   "coin.holding": "आपके पास {amount} {symbol} है",
+   "coin.statsTitle": "मार्केट आँकड़े",
+   "coin.aboutTitle": "परिचय",
+   "coin.aboutUnavailable": "अभी कोई विवरण उपलब्ध नहीं है।",
+   "coin.marketCap": "मार्केट कैप",
+   "coin.volume24h": "24 घंटे का वॉल्यूम",
+   "coin.high24h": "24 घंटे का उच्चतम",
+   "coin.low24h": "24 घंटे का न्यूनतम",
+   "coin.supply": "प्रचलित आपूर्ति",

```

```

+ "coin.ath": "सर्वकालिक उच्चतम",
+ "coin.viewCoinGecko": "CoinGecko पर देखें",
+ "coin.loadError": "अभी इस कॉइन की और जानकारी लोड नहीं हो सकी।",
+ "coin.chartUnavailable": "चार्ट अभी उपलब्ध नहीं है।",
+ "account.title": "साइन इन करें",
+ "account.intro": "वैकल्पिक। खाता आपकी डिस्प्ले मुद्रा, भाषा और वॉचलिस्ट को डिवाइसों के बीच सिंक
+ "account.tabSignIn": "साइन इन करें",
+ "account.tabCreate": "खाता बनाएं",
+ "account.usernameLabel": "उपयोगकर्ता नाम",
+ "account.usernameHint": "3-24 वर्ण: अक्षर, अंक या अंडरस्कोर।",
+ "account.passwordLabel": "पासवर्ड",
+ "account.confirmLabel": "पासवर्ड की पुष्टि करें",
+ "account.walletWarning": "यह पासवर्ड आपका वॉलेट पुनर्प्राप्त नहीं कर सकता। अपना रिकवरी फ्रेज़ सु
+ "account.signInSubmit": "साइन इन करें",
+ "account.createSubmit": "खाता बनाएं",
+ "account.working": "कार्य जारी है...",
+ "account.synced": "सिंक हो गया।",
+ "account.signedInAs": "{username} के रूप में साइन इन",
+ "account.syncNote": "सिंकिंग: डिस्प्ले मुद्रा, भाषा और वॉचलिस्ट।",
+ "account.signOut": "साइन आउट करें",
+ "account.deleteLabel": "खाता हटाएं",
+ "account.deletePlaceholder": "पुष्टि हेतु पासवर्ड",
+ "account.deleteBtn": "खाता हटाएं",
+ "account.deleteNeedsPassword": "अपना खाता हटाने के लिए पासवर्ड दर्ज करें।",
+ "account.deleteConfirm": "इससे आपका खाता और उसकी सिंक की गई सेटिंग्स हट जाएंगी। आपके व
+ "account.entryBtn": "साइन इन / खाता बनाएं",
+ "account.errUsername": "उपयोगकर्ता नाम 3-24 वर्णों का होना चाहिए: अक्षर, अंक या अंडरस्कोर।",
+ "account.errPasswordShort": "पासवर्ड कम से कम 10 वर्णों का होना चाहिए।",
+ "account.errMismatch": "पासवर्ड मेल नहीं खाते।",
+ "account.errBadLogin": "उपयोगकर्ता नाम या पासवर्ड गलत है।",
+ "account.errTaken": "वह उपयोगकर्ता नाम पहले से लिया जा चुका है।",
+ "account.errRateLimited": "बहुत अधिक प्रयास। कृपया प्रतीक्षा करें और फिर से प्रयास करें।",
+ "account.errUnavailable": "खाते अभी उपलब्ध नहीं हैं।",
+ "account.errGeneric": "कुछ गड़बड़ हो गई। कृपया फिर से प्रयास करें।",
+ "swap.customAddressOption": "अन्य टोकन (पता पेस्ट करें)...",

```

```

"buy.title": "क्रिप्टो खरीदें",
"buy.description1": "खरीदारी की सुविधा MoonPay द्वारा संचालित है, जो एक अलग थर्ड-पार्टी सेवा है
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/index
--- /tmp/orig/toKen-exchange-main/index.html      2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/index.html      2026-09-19 17:07:06.108723801 +0000
@@ -69,6 +69,7 @@
</div>
<h2 data-il8n="onboarding.title">Welcome</h2>
<p class="muted" data-il8n="onboarding.subtitle">Set up your wallet to get s
+ <p><button id="btn-onboarding-signin" type="button" class="link hidden" data
<div class="warn onboarding-intro-card" style="text-align:left; margin: 10px

```

```

    This is the standalone <strong>website</strong> version of Token Exchange.
    encrypts your wallet and stores it only in this browser (nowhere else) --
@@ -324,12 +325,54 @@
    <section id="screen-prices" class="screen hidden">
        <h2 data-il8n="common.livePrices">Live prices</h2>
        <p class="muted" data-il8n="prices.description">USD prices from CoinGecko's
+
        <div class="prices-tabs" role="tablist">
+
            <button type="button" class="prices-tab active" data-prices-tab="crypto" d
+
            <button type="button" class="prices-tab" data-prices-tab="currencies" data
+
        </div>
+
        <input type="search" id="prices-search" class="prices-search" data-il8n-plac
+
        <p id="prices-tab-note" class="muted hidden" data-il8n="prices.currenciesNot
        <div id="prices-list" class="prices-list"></div>
        <p id="prices-error" class="error hidden"></p>
        <p id="prices-status" class="hint hidden"></p>
        <button class="link back-btn" data-back="screen-main" data-il8n="common.back
    </section>

+
+ <!-- COIN DETAIL (opened from a Live prices row) -->
+
+ <section id="screen-coin" class="screen hidden">
+
+     <div class="coin-head">
+
+         <span id="coin-icon" class="coin-icon"></span>
+
+         <div class="coin-title">
+
+             <h2 id="coin-name"></h2>
+
+             <span id="coin-symbol" class="muted"></span>
+
+         </div>
+
+         <span id="coin-rank" class="coin-rank hidden"></span>
+
+     </div>
+
+     <div class="coin-price-block">
+
+         <div id="coin-price" class="coin-price"></div>
+
+         <span id="coin-change" class="price-change hidden"></span>
+
+     </div>
+
+     <div id="coin-chart" class="coin-chart"></div>
+
+     <p id="coin-chart-readout" class="muted small coin-chart-readout"></p>
+
+     <div class="coin-ranges" role="tablist">
+
+         <button type="button" class="coin-range" data-days="1">24H</button>
+
+         <button type="button" class="coin-range active" data-days="7">7D</button>
+
+         <button type="button" class="coin-range" data-days="30">1M</button>
+
+         <button type="button" class="coin-range" data-days="365">1Y</button>
+
+     </div>
+
+     <div class="coin-swap-card">
+
+         <p id="coin-holding" class="muted small hidden"></p>
+
+         <button type="button" id="btn-coin-swap" class="primary"></button>
+
+         <p id="coin-swap-note" class="hint hidden"></p>
+
+     </div>
+
+     <h3 data-il8n="coin.statsTitle">Market stats</h3>
+
+     <div id="coin-stats" class="coin-stats"></div>

```

```

+   <h3 data-il8n="coin.aboutTitle">About</h3>
+   <p id="coin-about" class="muted"></p>
+   <p><a id="coin-link-cg" class="link" target="_blank" rel="noopener noreferrer
+   <p id="coin-error" class="error hidden"></p>
+   <button id="coin-back" class="link back-btn" data-back="screen-prices" data-
+ </section>
+
+   <!-- PREDICTIONS (read-only) -->
+   <section id="screen-predictions" class="screen hidden">
+     <h2 data-il8n="predictions.title">Trending Markets</h2>
@@ -429,9 +472,44 @@
+     <p id="swap-error" class="error hidden"></p>
+   </section>
+
+   <!-- ACCOUNT (optional sign-in; syncs display settings only, never keys) -->
+   <section id="screen-account" class="screen hidden">
+     <h2 data-il8n="account.title">Sign in</h2>
+     <div id="account-signed-out">
+       <p class="muted" data-il8n="account.intro">Optional. An account syncs your
+       <div class="account-tabs" role="tablist">
+         <button type="button" class="account-tab active" data-account-mode="sign
+         <button type="button" class="account-tab" data-account-mode="create" dat
+       </div>
+       <label for="account-username" data-il8n="account.usernameLabel">Username</
+       <input type="text" id="account-username" autocomplete="username" autocapit
+       <p id="account-username-hint" class="muted small hidden" data-il8n="accoun
+       <label for="account-password" data-il8n="account.passwordLabel">Password</
+       <input type="password" id="account-password" autocomplete="current-passwor
+       <div id="account-confirm-row" class="hidden">
+         <label for="account-password2" data-il8n="account.confirmLabel">Confirm
+         <input type="password" id="account-password2" autocomplete="new-password
+         <div class="warn small" data-il8n="account.walletWarning">This password
+       </div>
+       <button id="btn-account-submit" class="primary" data-il8n="account.signInS
+     </div>
+     <div id="account-signed-in" class="hidden">
+       <p id="account-signed-in-as"></p>
+       <p class="muted small" data-il8n="account.syncNote">Syncing: display curre
+       <button id="btn-account-signout" class="secondary" data-il8n="account.sign
+       <label for="account-delete-password" data-il8n="account.deleteLabel">Delet
+       <input type="password" id="account-delete-password" autocomplete="current-
+       <button id="btn-account-delete" class="danger" data-il8n="account.deleteBt
+     </div>
+     <p id="account-status" class="hint hidden"></p>
+     <p id="account-error" class="error hidden"></p>
+     <button id="account-back" class="link back-btn" data-back="screen-settings"
+   </section>

```

```

+
+ <!-- SETTINGS -->
+ <section id="screen-settings" class="screen hidden">
+   <h2 data-il8n="settings.title">Settings</h2>
+   <button id="btn-goto-account" class="secondary hidden" data-il8n="account.en
+   <button id="btn-add-account" class="secondary" data-il8n="settings.addAccoun
+   <button id="btn-goto-import-key" class="secondary" data-il8n="settings.impor
+   <button id="btn-goto-add-watch" class="secondary" data-il8n="settings.addWat
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/ja.js
--- /tmp/orig/toKen-exchange-main/ja.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/ja.js 2026-09-24 23:22:06.591109886 +0000
@@ -73,6 +73,61 @@
    "prices.description": "CoinGecko の公開 API による米ドル建て価格です。あくまで参考情
    "prices.loading": "読み込み中...",
    "prices.naText": "該当なし",
+   "prices.tabCrypto": "暗号資産",
+   "prices.tabCurrencies": "通貨",
+   "prices.searchPlaceholder": "検索...",
+   "prices.currenciesNote": "各通貨 1 単位が、表示通貨でいくらになるかを示します。",
+   "prices.noResults": "結果がありません。",
+   "prices.viewCoin": "{name} を見る",
+   "coin.swapBtn": "{symbol} にスワップ",
+   "coin.swapHint": "次の画面で、保有資産のうちどれで支払うかを選べます。",
+   "coin.swapNeedsWallet": "スワップするにはウォレットのロックを解除してください。",
+   "coin.swapNoRouter": "このネットワークではまだスワップを利用できません。",
+   "coin.swapUnavailableNetwork": "{network} では {symbol} をまだスワップできません。",
+   "coin.holding": "{amount} {symbol} を保有しています",
+   "coin.statsTitle": "マーケット情報",
+   "coin.aboutTitle": "概要",
+   "coin.aboutUnavailable": "現在、説明はありません。",
+   "coin.marketCap": "時価総額",
+   "coin.volume24h": "24時間取引量",
+   "coin.high24h": "24時間高値",
+   "coin.low24h": "24時間安値",
+   "coin.supply": "流通供給量",
+   "coin.ath": "過去最高値",
+   "coin.viewCoingecko": "CoinGecko で見る",
+   "coin.loadError": "現在、このコインの詳細を読み込めませんでした。",
+   "coin.chartUnavailable": "現在、チャートを表示できません。",
+   "account.title": "サインイン",
+   "account.intro": "任意機能です。アカウントは表示通貨・言語・ウォッチリストをデバイス間で
+   "account.tabSignIn": "サインイン",
+   "account.tabCreate": "アカウントを作成",
+   "account.usernameLabel": "ユーザー名",
+   "account.usernameHint": "3~24文字: 英字、数字、アンダースコア。",
+   "account.passwordLabel": "パスワード",
+   "account.confirmLabel": "パスワード (確認)",

```

```

+   "account.walletWarning": "このパスワードではウォレットを復元できません。リカバリーフレ
+   "account.signInSubmit": "サインイン",
+   "account.createSubmit": "アカウントを作成",
+   "account.working": "処理中...",
+   "account.synced": "同期しました。",
+   "account.signedInAs": "{username} としてサインイン中",
+   "account.syncNote": "同期中: 表示通貨、言語、ウォッチリスト。",
+   "account.signOut": "サインアウト",
+   "account.deleteLabel": "アカウントを削除",
+   "account.deletePlaceholder": "確認用パスワード",
+   "account.deleteBtn": "アカウントを削除",
+   "account.deleteNeedsPassword": "アカウントを削除するにはパスワードを入力してください。
+   "account.deleteConfirm": "アカウントと同期設定が削除されます。ウォレットと資金には影響
+   "account.entryBtn": "サインイン / アカウント作成",
+   "account.errUsername": "ユーザー名は3~24文字 (英字、数字、アンダースコア) にしてくださ
+   "account.errPasswordShort": "パスワードは10文字以上にしてください。",
+   "account.errMismatch": "パスワードが一致しません。",
+   "account.errBadLogin": "ユーザー名またはパスワードが正しくありません。",
+   "account.errTaken": "そのユーザー名は既に使用されています。",
+   "account.errRateLimited": "試行回数が多すぎます。しばらくしてからもう一度お試しください
+   "account.errUnavailable": "現在アカウント機能は利用できません。",
+   "account.errGeneric": "問題が発生しました。もう一度お試しください。",
+   "swap.customAddressOption": "その他のトークン (アドレスを貼り付け) ...",

    "buy.title": "暗号資産を購入",
    "buy.description1": "購入機能は、独立した第三者サービスである MoonPay によって提供さ
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/packa
--- /tmp/orig/toKen-exchange-main/package.json 2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/package.json 2026-09-19 17:05:19.363683745 +0000
@@ -10,5 +10,8 @@
    "ethers": "5.7.2",
    "@walletconnect/sign-client": "2.24.0",
    "qrcode-generator": "2.0.4"
+ },
+ "engines": {
+   "node": ">=22.13.0"
+ }
}

diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/popup
--- /tmp/orig/toKen-exchange-main/popup.css 2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/popup.css 2026-09-19 16:54:09.047916742 +0000
@@ -899,3 +899,112 @@
    html[dir="rtl"] input[type="password"],
    html[dir="rtl"] textarea { text-align: right; }
    html[dir="rtl"] .mono { direction: ltr; unicode-bidi: embed; text-align: right;
+
+ /* Prices screen: Crypto / Currencies tabs + search */

```

```

+.prices-tabs {
+  display: flex; gap: 6px; margin: 10px 0 8px;
+  background: var(--bg-inset); border: 1px solid var(--line);
+  border-radius: 10px; padding: 4px;
+}
+.prices-tab {
+  flex: 1; background: transparent; border: 0; border-radius: 7px;
+  color: var(--text-muted); font: inherit; font-weight: 600; font-size: 13px;
+  padding: 7px 10px; cursor: pointer;
+}
+.prices-tab:hover { color: var(--text); }
+.prices-tab.active { background: var(--gold); color: var(--gold-ink); }
+.prices-tab:focus-visible { outline: 2px solid var(--gold-bright); outline-offse
+.prices-search {
+  width: 100%; box-sizing: border-box; margin: 0 0 8px;
+  background: var(--bg-inset); border: 1px solid var(--line); border-radius: 8px
+  color: var(--text); font: inherit; font-size: 13px; padding: 9px 12px;
+}
+.prices-search:focus { outline: none; border-color: var(--gold); }
+#prices-tab-note { margin: 0 0 8px; font-size: 12px; }
+
+/* Prices screen polish: symbol stacked under the name, price with a 24h
+  change pill stacked on the right, star in its own fixed column so every
+  row lines up regardless of how long the price text is. */
+.price-left { flex: 1; min-width: 0; }
+.price-id { display: flex; flex-direction: column; min-width: 0; line-height: 1.
+.price-id .price-name { overflow: hidden; text-overflow: ellipsis; white-space:
+.price-row .price-id .price-symbol,
+html[dir="rtl"] .price-row .price-id .price-symbol { margin: 0; font-size: 11px;
+.price-right { flex-shrink: 0; gap: 6px; }
+.price-quote { display: flex; flex-direction: column; align-items: flex-end; gap
+.price-quote .price-change,
+html[dir="rtl"] .price-quote .price-change {
+  margin: 0; padding: 1px 7px; border-radius: 999px;
+  font-size: 10.5px; font-weight: 600; line-height: 1.5;
+}
+.price-quote .price-change.up { background: rgba(127, 201, 154, 0.14); }
+.price-quote .price-change.down { background: rgba(226, 104, 90, 0.14); }
+.price-right .star-btn {
+  display: flex; align-items: center; justify-content: center;
+  width: 26px; height: 32px; padding: 0; font-size: 16px;
+}
+.prices-search {
+  padding-inline-start: 34px;
+  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000
+  background-repeat: no-repeat; background-position: 11px center; background-siz
+}

```

```

+html[dir="rtl"] .prices-search { background-position: right 11px center; }
+.prices-tab.active { box-shadow: 0 1px 6px rgba(0, 0, 0, 0.35); }
+/* Keep numbers and +/- signs reading correctly inside right-to-left layouts. */
+.price-quote .price-change, .price-quote .price-usd { direction: ltr; unicode-bi
+
+/* Coin rows are links to the coin's CoinGecko page (star stays a button). */
+.price-link {
+  display: flex; align-items: center; justify-content: space-between;
+  flex: 1; min-width: 0; gap: 10px; color: inherit; text-decoration: none;
+  cursor: pointer;
+}
+.price-link:hover .price-name { text-decoration: underline; text-underline-offse
+.price-link:focus-visible { outline: 2px solid var(--gold-bright); outline-offse
+.price-link { margin-inline-end: 6px; }
+
+/* ---- Coin detail screen ---- */
+.coin-head { display: flex; align-items: center; gap: 12px; margin-bottom: 6px;
+.coin-head .coin-icon .token-icon, .coin-head .coin-icon img { width: 40px; heig
+.coin-title { flex: 1; min-width: 0; }
+.coin-title h2 { margin: 0; line-height: 1.15; }
+.coin-rank {
+  font-size: 11px; font-weight: 700; padding: 3px 8px; border-radius: 999px;
+  background: var(--bg-inset); border: 1px solid var(--line); color: var(--text-
+}
+.coin-price-block { display: flex; align-items: center; gap: 10px; margin: 6px 0
+.coin-price { font-size: 28px; font-weight: 700; color: var(--gold-bright); dire
+.coin-price-block .price-change { font-size: 12px; padding: 2px 8px; border-radi
+.coin-price-block .price-change.up { background: rgba(127, 201, 154, 0.14); }
+.coin-price-block .price-change.down { background: rgba(226, 104, 90, 0.14); }
+.coin-chart { height: 130px; margin: 6px 0 2px; color: var(--text-muted); touch-
+.coin-chart.up { color: #7fc99a; }
+.coin-chart.down { color: #e2685a; }
+.coin-chart svg { width: 100%; height: 100%; display: block; }
+.coin-chart-msg { display: flex; align-items: center; justify-content: center; h
+.coin-chart-readout { min-height: 18px; margin: 2px 0 8px; direction: ltr; unico
+.coin-ranges {
+  display: flex; gap: 6px; margin: 0 0 14px;
+  background: var(--bg-inset); border: 1px solid var(--line); border-radius: 10p
+}
+.coin-range {
+  flex: 1; background: transparent; border: 0; border-radius: 7px; color: var(--
+  font: inherit; font-weight: 600; font-size: 12px; padding: 6px 4px; cursor: po
+}
+.coin-range.active { background: var(--gold); color: var(--gold-ink); }
+.coin-swap-card {
+  background: var(--bg-inset); border: 1px solid var(--line); border-radius: 12p
+  padding: 12px; margin: 0 0 16px;

```

```

+}
+.coin-swap-card p { margin: 0 0 8px; }
+.coin-swap-card button { width: 100%; }
+.coin-swap-card button:disabled { opacity: 0.5; cursor: not-allowed; }
+.coin-stats { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; margin-bo
+.coin-stat {
+  background: var(--bg-inset); border: 1px solid var(--line); border-radius: 10p
+  padding: 8px 10px; min-width: 0;
+}
+.coin-stat .k { display: block; font-size: 11px; color: var(--text-muted); margi
+.coin-stat .v { display: block; font-size: 13.5px; font-weight: 600; overflow: h
+/* Price rows now open the coin screen, so they are buttons, not anchors. */
+button.price-link { background: none; border: 0; padding: 0; font: inherit; text
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/popup
--- /tmp/orig/toKen-exchange-main/popup.html      2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/popup.html      2026-09-24 23:19:17.974492317 +0000
@@ -161,6 +161,7 @@
    </div>
    <h2 data-il8n="onboarding.title">Welcome</h2>
    <p class="muted" data-il8n="onboarding.subtitle">Set up your wallet to get s
+  <p><button id="btn-onboarding-signin" type="button" class="link hidden" data
    <button id="btn-goto-create" class="primary" data-il8n="onboarding.createBtn
    <button id="btn-goto-import" class="secondary" data-il8n="onboarding.importB
    <button id="btn-goto-support-onboarding" class="link" data-il8n="common.need
@@ -352,12 +353,54 @@
    <section id="screen-prices" class="screen hidden">
      <h2 data-il8n="common.livePrices">Live prices</h2>
      <p class="muted" data-il8n="prices.description">USD prices from CoinGecko's
+    <div class="prices-tabs" role="tablist">
+      <button type="button" class="prices-tab active" data-prices-tab="crypto" d
+      <button type="button" class="prices-tab" data-prices-tab="currencies" data
+    </div>
+    <input type="search" id="prices-search" class="prices-search" data-il8n-plac
+    <p id="prices-tab-note" class="muted hidden" data-il8n="prices.currenciesNot
    <div id="prices-list" class="prices-list"></div>
    <p id="prices-error" class="error hidden"></p>
    <p id="prices-status" class="hint hidden"></p>
    <button class="link back-btn" data-back="screen-main" data-il8n="common.back
  </section>

+  <!-- COIN DETAIL (opened from a Live prices row) -->
+  <section id="screen-coin" class="screen hidden">
+    <div class="coin-head">
+      <span id="coin-icon" class="coin-icon"></span>
+      <div class="coin-title">
+        <h2 id="coin-name"></h2>
+        <span id="coin-symbol" class="muted"></span>

```

```

+     </div>
+     <span id="coin-rank" class="coin-rank hidden"></span>
+ </div>
+ <div class="coin-price-block">
+     <div id="coin-price" class="coin-price"></div>
+     <span id="coin-change" class="price-change hidden"></span>
+ </div>
+ <div id="coin-chart" class="coin-chart"></div>
+ <p id="coin-chart-readout" class="muted small coin-chart-readout"></p>
+ <div class="coin-ranges" role="tablist">
+     <button type="button" class="coin-range" data-days="1">24H</button>
+     <button type="button" class="coin-range active" data-days="7">7D</button>
+     <button type="button" class="coin-range" data-days="30">1M</button>
+     <button type="button" class="coin-range" data-days="365">1Y</button>
+ </div>
+ <div class="coin-swap-card">
+     <p id="coin-holding" class="muted small hidden"></p>
+     <button type="button" id="btn-coin-swap" class="primary"></button>
+     <p id="coin-swap-note" class="hint hidden"></p>
+ </div>
+ <h3 data-il8n="coin.statsTitle">Market stats</h3>
+ <div id="coin-stats" class="coin-stats"></div>
+ <h3 data-il8n="coin.aboutTitle">About</h3>
+ <p id="coin-about" class="muted"></p>
+ <p><a id="coin-link-cg" class="link" target="_blank" rel="noopener noreferrer">
+ <p id="coin-error" class="error hidden"></p>
+ <button id="coin-back" class="link back-btn" data-back="screen-prices" data-
+ </section>
+
+ <!-- PREDICTIONS (read-only) -->
+ <section id="screen-predictions" class="screen hidden">
+     <h2 data-il8n="predictions.title">Trending Markets</h2>
@@ -444,8 +487,44 @@
+ </section>
+
+ <!-- SETTINGS -->
+ <!-- ACCOUNT (optional sign-in; syncs display settings only, never keys) -->
+ <section id="screen-account" class="screen hidden">
+     <h2 data-il8n="account.title">Sign in</h2>
+     <div id="account-signed-out">
+         <p class="muted" data-il8n="account.intro">Optional. An account syncs your
+         <div class="account-tabs" role="tablist">
+             <button type="button" class="account-tab active" data-account-mode="sign
+             <button type="button" class="account-tab" data-account-mode="create" dat
+         </div>
+         <label for="account-username" data-il8n="account.usernameLabel">Username</
+         <input type="text" id="account-username" autocomplete="username" autocapit

```

```

+     <p id="account-username-hint" class="muted small hidden" data-il8n="accoun
+     <label for="account-password" data-il8n="account.passwordLabel">Password</
+     <input type="password" id="account-password" autocomplete="current-passwor
+     <div id="account-confirm-row" class="hidden">
+         <label for="account-password2" data-il8n="account.confirmLabel">Confirm
+         <input type="password" id="account-password2" autocomplete="new-password
+         <div class="warn small" data-il8n="account.walletWarning">This password
+     </div>
+     <button id="btn-account-submit" class="primary" data-il8n="account.signInS
+ </div>
+ <div id="account-signed-in" class="hidden">
+     <p id="account-signed-in-as"></p>
+     <p class="muted small" data-il8n="account.syncNote">Syncing: display curre
+     <button id="btn-account-signout" class="secondary" data-il8n="account.sign
+     <label for="account-delete-password" data-il8n="account.deleteLabel">Delet
+     <input type="password" id="account-delete-password" autocomplete="current-
+     <button id="btn-account-delete" class="danger" data-il8n="account.deleteBt
+ </div>
+ <p id="account-status" class="hint hidden"></p>
+ <p id="account-error" class="error hidden"></p>
+ <button id="account-back" class="link back-btn" data-back="screen-settings"
+ </section>
+
+ <!-- SETTINGS -->
+ <section id="screen-settings" class="screen hidden">
+     <h2 data-il8n="settings.title">Settings</h2>
+     <button id="btn-goto-account" class="secondary hidden" data-il8n="account.en
+     <button id="btn-add-account" class="secondary" data-il8n="settings.addAccoun
+     <button id="btn-goto-import-key" class="secondary" data-il8n="settings.impor
+     <button id="btn-goto-add-network" class="secondary" data-il8n="settings.addN
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/popup
--- /tmp/orig/toKen-exchange-main/popup.js      2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/popup.js      2026-09-24 23:21:03.286059270 +0000
@@ -38,7 +38,7 @@
    }

    function formatCurrency(amount) {
-    return `${currencySymbol()}${amount.toLocaleString(undefined, { minimumFractio
+    return TM_PRICES.formatMoney(amount, currentCurrency);
    }

    // ----- NETWORK COLO
@@ -587,6 +587,7 @@
    if (watchlistSymbols.has(key)) watchlistSymbols.delete(key);
    else watchlistSymbols.add(key);
    chrome.storage.local.set({ [TM_WATCHLIST_KEY]: Array.from(watchlistSymbols) })
+    scheduleAccountPush();

```

```

    }

    // Stable sort (guaranteed by the spec for Array#sort) -- starred coins move
@@ -601,7 +602,7 @@
    const priceText =
      c.price == null
        ? TM_I18N.t("prices.naText")
-      : `${currencySymbol()}${c.price.toLocaleString(undefined, { minimumFractio
+      : TM_PRICES.formatMoney(c.price, currentCurrency, { price: true });
    let changeHtml = "";
    if (typeof c.change24h === "number") {
      const cls = c.change24h >= 0 ? "up" : "down";
@@ -609,10 +610,16 @@
      changeHtml = `

```

```

+   <span class="price-right"><span class="price-quote"><span class="price-usd">
+   `;
+   return row;
+}
+
+function renderPricesList() {
+   const list = $("prices-list");
+   const q = ($("prices-search").value || "").trim().toLowerCase();
+   list.innerHTML = "";
+   let count = 0;
+   if (pricesTab === "crypto") {
+       sortByWatchlist(pricesBoardData).forEach((c) => {
+           if (q && !(c.name.toLowerCase().includes(q) || c.symbol.toLowerCase().incl
+               list.appendChild(renderPriceRow(c));
+               count++;
+           });
+       } else {
+           pricesRatesData.forEach((r) => {
+               if (q && !(r.name.toLowerCase().includes(q) || r.code.toLowerCase().includ
+               list.appendChild(renderCurrencyRow(r));
+               count++;
+           });
+       }
+       if (!count) {
+           const p = document.createElement("p");
+           p.className = "muted";
+           p.textContent = TM_I18N.t("prices.noResults");
+           list.appendChild(p);
+       }
+   }
+
+function setPricesTab(tab) {
+   pricesTab = tab;
+   document.querySelectorAll(".prices-tab").forEach((b) => b.classList.toggle("ac
+   $("prices-tab-note").classList.toggle("hidden", tab !== "currencies");
+   refreshPrices();
+}
+
+document.querySelectorAll(".prices-tab").forEach((b) => b.addEventListener("clie
+
+async function refreshPrices() {
+   hideError("prices-error");
+   $("prices-status").innerHTML = coinSpinnerHtml(TM_I18N.t("prices.loading"));
+   $("prices-status").classList.remove("hidden");
+   try {
+       const board = sortByWatchlist(await TM_PRICES.getPriceBoard(currentCurrency)

```

```

-   const list = $("prices-list");
-   list.innerHTML = "";
-   board.forEach((c) => list.appendChild(renderPriceRow(c)));
+   if (pricesTab === "crypto") pricesBoardData = await TM_PRICES.getPriceBoard(
+   else pricesRatesData = await TM_PRICES.getFiatRates(currentCurrency);
+   renderPricesList();
      $("prices-status").classList.add("hidden");
    } catch (e) {
      $("prices-status").classList.add("hidden");
@@ -637,6 +696,573 @@
    }
  }

+// ----- COIN DETAIL
+// The screen a Live prices row opens (same idea as MetaMask's token page):
+// price + 24h change, a price chart with a few time ranges, market stats, a
+// short description, and a Swap button. Swap here means "swap INTO this
+// coin": it is only offered when the coin exists on the selected network as
+// the native coin or as a token the person has added (matched by symbol) --
+// this wallet never guesses a token contract address. The Swap screen then
+// lets them choose which of their own holdings to pay with.
+let coinDetail = { coin: null, days: 7, gen: 0, chartGen: 0, chartPoints: [], sw
+
+function normalizeCoinSymbol(s) {
+  const up = String(s || "").toUpperCase();
+  return up === "MATIC" ? "POL" : up; // Polygon's native coin was renamed
+}
+
+// Native coin + tracked tokens on the selected network, with balances.
+async function getHeldAssets() {
+  const assets = [];
+  if (!currentStatus || !currentStatus.selectedAddress || !currentNetwork) retur
+  try {
+    const bal = await sendMsg("TM_GET_BALANCE", { address: currentStatus.selecte
+    assets.push({
+      key: "native", address: "", symbol: bal.symbol || currentNetwork.nativeCur
+      decimals: bal.decimals, balance: ethers.utils.formatUnits(bal.balanceWei,
+    });
+  } catch (e) {
+    assets.push({ key: "native", address: "", symbol: currentNetwork.nativeCurre
+  }
+  try {
+    const res = await sendMsg("TM_GET_TRACKED_TOKEN_BALANCES");
+    (res.tokens || []).forEach((t) => {
+      if (t.error) return;
+      assets.push({
+        key: t.address, address: t.address, symbol: t.symbol, decimals: t.decima

```

```

+         balance: ethers.utils.formatUnits(t.balanceWei, t.decimals),
+     });
+ });
+ } catch (e) { /* best-effort: native coin only */ }
+ return assets;
+}
+
+function assetLabel(a) {
+  const n = Number(a.balance);
+  return `${a.symbol} (${isFinite(n) ? n.toLocaleString(undefined, { maximumFrac
+}
+
+// ---- Swap screen asset pickers (From = what you hold, To = any of your assets
+let swapPopulateGen = 0;
+
+function syncSwapAsset(side) {
+  const sel = `${swap}-${side}-select`;
+  const input = `${swap}-${side}-custom`;
+  if (sel.value === "custom") {
+    input.classList.remove("hidden");
+    input.value = "";
+  } else {
+    input.classList.add("hidden");
+    input.value = sel.value === "native" ? "" : sel.value; // blank = native coi
+  }
+  `("swap-quote-display").classList.add("hidden"); // a changed pair invalidates
+}
+
+async function populateSwapSelects(pre) {
+  const myGen = ++swapPopulateGen;
+  const held = await getHeldAssets();
+  if (myGen !== swapPopulateGen) return;
+  const fromSel = `("swap-from-select");
+  const toSel = `("swap-to-select");
+  const toKey = pre && pre.toKey;
+  const fill = (sel, items) => {
+    sel.innerHTML = "";
+    items.forEach((a) => {
+      const o = document.createElement("option");
+      o.value = a.key;
+      o.textContent = assetLabel(a);
+      sel.appendChild(o);
+    });
+    const custom = document.createElement("option");
+    custom.value = "custom";
+    custom.textContent = TM_I18N.t("swap.customAddressOption");
+    sel.appendChild(custom);

```

```

+   };
+   // "From" only offers what the person actually holds (plus a paste-address
+   // escape hatch); the coin being bought is never offered as its own source.
+   let fromItems = held.filter((a) => Number(a.balance) > 0 && a.key !== toKey);
+   if (!fromItems.length) fromItems = held.filter((a) => a.key === "native" && a.
+   fill(fromSel, fromItems);
+   fill(toSel, held);
+   if (toKey && held.some((a) => a.key === toKey)) toSel.value = toKey;
+   else toSel.value = (held.find((a) => a.key !== fromSel.value) || held[0] || {
+   syncSwapAsset("from");
+   syncSwapAsset("to");
+ }
+
+ ["from", "to"].forEach((side) => {
+   $(`swap-${side}-select`).addEventListener("change", () => syncSwapAsset(side))
+ });
+
+ // ---- Coin screen
+ function renderCoinPrice(price, change) {
+   $("coin-price").textContent = price == null ? TM_I18N.t("prices.naText") : TM_
+   const el = $("coin-change");
+   if (typeof change === "number") {
+     el.className = `price-change ${change >= 0 ? "up" : "down"}`;
+     el.textContent = `${change >= 0 ? "+" : ""}${change.toFixed(2)}%`;
+   } else {
+     el.className = "price-change hidden";
+   }
+ }
+
+ function coinStat(label, value) {
+   const d = document.createElement("div");
+   d.className = "coin-stat";
+   const k = document.createElement("span");
+   k.className = "k";
+   k.textContent = label;
+   const v = document.createElement("span");
+   v.className = "v";
+   v.textContent = value;
+   d.appendChild(k);
+   d.appendChild(v);
+   return d;
+ }
+
+ async function openCoinDetail(c, fromScreen) {
+   const gen = ++coinDetail.gen;
+   coinDetail.coin = c;
+   coinDetail.swapTarget = null;

```

```

+   $("coin-back").dataset.back = fromScreen || "screen-prices";
+   hideError("coin-error");
+   $("coin-icon").innerHTML = tokenIconHtml(c.symbol);
+   $("coin-name").textContent = c.name;
+   $("coin-symbol").textContent = c.symbol;
+   $("coin-rank").classList.add("hidden");
+   renderCoinPrice(c.price, c.change24h);
+   $("coin-stats").innerHTML = "";
+   $("coin-about").textContent = TM_I18N.t("prices.loading");
+   $("coin-link-cg").href = c.url || "https://www.coingecko.com/";
+   const btn = $("btn-coin-swap");
+   btn.textContent = TM_I18N.t("coin.swapBtn", { symbol: c.symbol });
+   btn.disabled = true;
+   $("coin-holding").classList.add("hidden");
+   $("coin-swap-note").classList.add("hidden");
+   document.querySelectorAll(".coin-range").forEach((b) => b.classList.toggle("ac
+   showScreen("screen-coin");
+   loadCoinChart(gen);
+   loadCoinInfo(gen);
+   loadCoinSwapState(gen);
+}
+
+async function loadCoinInfo(gen) {
+   const c = coinDetail.coin;
+   try {
+       const d = await TM_PRICES.getCoinDetail(c.symbol, currentCurrency);
+       if (gen !== coinDetail.gen) return;
+       if (d.price !== null) renderCoinPrice(d.price, d.change24h !== null ? d.change
+       if (d.rank !== null) {
+           $("coin-rank").textContent = `#${d.rank}`;
+           $("coin-rank").classList.remove("hidden");
+       }
+       const stats = $("coin-stats");
+       stats.innerHTML = "";
+       const fm = (n) => TM_PRICES.formatMoney(n, currentCurrency, { price: true })
+       if (d.marketCap !== null) stats.appendChild(coinStat(TM_I18N.t("coin.marketCa
+       if (d.volume !== null) stats.appendChild(coinStat(TM_I18N.t("coin.volume24h")
+       if (d.high24h !== null) stats.appendChild(coinStat(TM_I18N.t("coin.high24h"),
+       if (d.low24h !== null) stats.appendChild(coinStat(TM_I18N.t("coin.low24h"), f
+       if (d.circulatingSupply !== null) stats.appendChild(coinStat(TM_I18N.t("coin.
+       if (d.ath !== null) {
+           const pct = d.athChange !== null ? ` (${d.athChange.toFixed(1)}%)` : "";
+           stats.appendChild(coinStat(TM_I18N.t("coin.ath"), fm(d.ath) + pct));
+       }
+       $("coin-about").textContent = d.description || TM_I18N.t("coin.aboutUnavaila
+   } catch (e) {
+       if (gen !== coinDetail.gen) return;

```

```

+     $("coin-about").textContent = TM_I18N.t("coin.aboutUnavailable");
+     showError("coin-error", TM_I18N.t("coin.loadError"));
+ }
+}
+
+async function loadCoinChart(gen) {
+  const c = coinDetail.coin;
+  const token = ++coinDetail.chartGen;
+  const box = $("coin-chart");
+  box.className = "coin-chart";
+  box.innerHTML = `<div class="coin-chart-msg">${escapeHtml(TM_I18N.t("prices.lo
+  $("coin-chart-readout").textContent = "";
+  try {
+    const pts = await TM_PRICES.getCoinChart(c.symbol, currentCurrency, coinDeta
+    if (gen !== coinDetail.gen || token !== coinDetail.chartGen) return;
+    drawCoinChart(pts);
+  } catch (e) {
+    if (gen !== coinDetail.gen || token !== coinDetail.chartGen) return;
+    box.innerHTML = `<div class="coin-chart-msg">${escapeHtml(TM_I18N.t("coin.ch
+  }
+}
+
+function coinRangeSummary() {
+  const pts = coinDetail.chartPoints;
+  if (pts.length < 2) return "";
+  const first = pts[0][1];
+  const last = pts[pts.length - 1][1];
+  const pct = first ? ((last - first) / first) * 100 : 0;
+  const label = { 1: "24H", 7: "7D", 30: "1M", 365: "1Y" }[coinDetail.days] || "
+  return `${label}: ${pct >= 0 ? "+" : ""}${pct.toFixed(2)}%`;
+}
+
+function drawCoinChart(pts) {
+  const box = $("coin-chart");
+  coinDetail.chartPoints = pts;
+  if (!pts || pts.length < 2) {
+    box.innerHTML = `<div class="coin-chart-msg">${escapeHtml(TM_I18N.t("coin.ch
+    return;
+  }
+  const W = 320, H = 130, PAD = 6;
+  const prices = pts.map((p) => p[1]);
+  const min = Math.min(...prices);
+  const max = Math.max(...prices);
+  const span = max - min || 1;
+  const xs = pts.map((_, i) => (i / (pts.length - 1)) * W);
+  const ys = prices.map((p) => H - PAD - ((p - min) / span) * (H - PAD * 2));
+  const line = xs.map((x, i) => `${i ? "L" : "M"}${x.toFixed(1)} ${ys[i].toFixed

```

```

+   const area = `${line} L${W} ${H} L0 ${H} Z`;
+   box.className = `coin-chart ${prices[prices.length - 1] >= prices[0] ? "up" :
+   box.innerHTML = `<svg viewBox="0 0 ${W} ${H}" preserveAspectRatio="none" role=
+     <path d="${area}" fill="currentColor" fill-opacity="0.12" stroke="none"></pa
+     <path d="${line}" fill="none" stroke="currentColor" stroke-width="2" stroke-
+     <line id="coin-chart-cursor" x1="0" y1="0" x2="0" y2="${H}" stroke="currentC
+   </svg>`;
+   $("coin-chart-readout").textContent = coinRangeSummary();
+ }
+
+ // Drag/hover over the chart to read the price at that moment.
+ (function wireCoinChartScrub() {
+   const box = $("coin-chart");
+   const show = (e) => {
+     const pts = coinDetail.chartPoints;
+     if (!pts || pts.length < 2) return;
+     const rect = box.getBoundingClientRect();
+     if (!rect.width) return;
+     const ratio = Math.min(1, Math.max(0, (e.clientX - rect.left) / rect.width));
+     const idx = Math.round(ratio * (pts.length - 1));
+     const [ts, price] = pts[idx];
+     const cursor = $("coin-chart-cursor");
+     if (cursor) {
+       const x = (idx / (pts.length - 1)) * 320;
+       cursor.setAttribute("x1", x);
+       cursor.setAttribute("x2", x);
+       cursor.setAttribute("opacity", "0.7");
+     }
+     const when = new Date(ts).toLocaleString(undefined, coinDetail.days === 1
+       ? { hour: "numeric", minute: "2-digit" }
+       : coinDetail.days <= 7 ? { month: "short", day: "numeric", hour: "numeric"
+     $("coin-chart-readout").textContent = `${TM_PRICES.formatMoney(price, curren
+   };
+   const reset = () => {
+     const cursor = $("coin-chart-cursor");
+     if (cursor) cursor.setAttribute("opacity", "0");
+     $("coin-chart-readout").textContent = coinRangeSummary();
+   };
+   box.addEventListener("pointermove", show);
+   box.addEventListener("pointerdown", show);
+   box.addEventListener("pointerleave", reset);
+   box.addEventListener("pointerup", reset);
+   box.addEventListener("pointercancel", reset);
+ }) ();
+
+ document.querySelectorAll(".coin-range").forEach((b) => {
+   b.addEventListener("click", () => {

```

```

+   coinDetail.days = Number(b.dataset.days);
+   document.querySelectorAll(".coin-range").forEach((x) => x.classList.toggle("
+   loadCoinChart(coinDetail.gen);
+   });
+});
+
+async function loadCoinSwapState(gen) {
+  const c = coinDetail.coin;
+  const btn = $("btn-coin-swap");
+  const note = $("coin-swap-note");
+  const showNote = (text) => { note.textContent = text; note.classList.remove("h
+  if (!currentNetwork || !currentStatus || !currentStatus.selectedAddress) {
+    showNote(TM_I18N.t("coin.swapNeedsWallet"));
+    return;
+  }
+  if (!currentNetwork.swapRouter) {
+    showNote(TM_I18N.t("coin.swapNoRouter"));
+    return;
+  }
+  const held = await getHeldAssets();
+  if (gen !== coinDetail.gen) return;
+  const want = normalizeCoinSymbol(c.symbol);
+  const match = held.find((a) => normalizeCoinSymbol(a.symbol) === want);
+  if (!match) {
+    showNote(TM_I18N.t("coin.swapUnavailableNetwork", { symbol: c.symbol, networ
+    return;
+  }
+  coinDetail.swapTarget = match;
+  if (Number(match.balance) > 0) {
+    const holding = $("coin-holding");
+    holding.textContent = TM_I18N.t("coin.holding", {
+      amount: Number(match.balance).toLocaleString(undefined, { maximumFractionD
+      symbol: match.symbol,
+    });
+    holding.classList.remove("hidden");
+  }
+  btn.disabled = false;
+  showNote(TM_I18N.t("coin.swapHint"));
+}
+
+$(".btn-coin-swap").addEventListener("click", () => {
+  const m = coinDetail.swapTarget;
+  if (!m) return;
+  setupSwapScreen({ toKey: m.key });
+  showScreen("screen-swap");
+});
+
+

```

```

+// ----- ACCOUNT (opt
+// Username sign-in for the website only. It syncs three display settings
+// (currency, language, watchlist) and nothing else -- never keys, phrases,
+// addresses or balances. See accounts-api.js for the server side. The
+// browser never sends the raw password: it derives an auth key from
+// (username, password) and sends that instead. The whole feature stays
+// hidden unless the server reports that accounts are switched on.
+let accountEnabled = false;
+let accountUser = null;
+let accountMode = "signin";
+let accountApplying = false; // true while applying pulled settings, so that doe
+let accountPushTimer = null;
+
+// EXTENSION NOTE: this popup runs at a chrome-extension:// origin, a
+// different origin from the wallet's website, so it cannot share the
+// website's session cookie or call its same-origin-only API. initAccount()
+// below simply finds no reachable API, accountEnabled stays false, and the
+// whole feature stays hidden here -- no separate flag needed. Real
+// extension support would need bearer-token cross-origin auth, not built
+// yet.
+
+async function accountApi(method, path, body) {
+  const res = await fetch(path, {
+    method,
+    credentials: "same-origin",
+    headers: Object.assign({ "X-TM-Requested-With": "web" }, body ? { "Content-T
+    body: body ? JSON.stringify(body) : undefined,
+  });
+  let json = null;
+  try { json = await res.json(); } catch (e) { /* non-JSON reply */ }
+  if (!res.ok) {
+    const err = new Error((json && json.error) || "server_error");
+    err.code = err.message;
+    err.status = res.status;
+    throw err;
+  }
+  return json;
+}
+
+async function deriveAccountAuthKey(username, password) {
+  const enc = new TextEncoder();
+  const material = await crypto.subtle.importKey("raw", enc.encode(password.norm
+  const bits = await crypto.subtle.deriveBits(
+    { name: "PBKDF2", hash: "SHA-256", salt: enc.encode("tm-account-v1:" + usern
+    material,
+    256
+  );

```

```

+   return Array.from(new Uint8Array(bits)).map((b) => b.toString(16).padStart(2,
+ })
+
+function accountErrorText(code) {
+   const map = {
+     bad_credentials: "account.errBadLogin",
+     username_taken: "account.errTaken",
+     rate_limited: "account.errRateLimited",
+     unavailable: "account.errUnavailable",
+     bad_request: "account.errUsername",
+   };
+   return TM_I18N.t(map[code] || "account.errGeneric");
+ }
+
+function updateAccountUi() {
+   const entry = $("btn-goto-account");
+   const onboardingLink = $("btn-onboarding-signin");
+   entry.classList.toggle("hidden", !accountEnabled);
+   onboardingLink.classList.toggle("hidden", !accountEnabled || !!accountUser);
+   if (accountUser) {
+     entry.removeAttribute("data-i18n"); // otherwise a language switch would ove
+     entry.textContent = TM_I18N.t("account.signedInAs", { username: accountUser
+   } else {
+     entry.setAttribute("data-i18n", "account.entryBtn");
+     entry.textContent = TM_I18N.t("account.entryBtn");
+   }
+   $("account-signed-out").classList.toggle("hidden", !!accountUser);
+   $("account-signed-in").classList.toggle("hidden", !accountUser);
+   $("account-signed-in-as").textContent = accountUser ? TM_I18N.t("account.signe
+   $("account-back").dataset.back = $("account-back").dataset.back || "screen-set
+ }
+
+function setAccountMode(mode) {
+   accountMode = mode;
+   document.querySelectorAll(".account-tab").forEach((b) => b.classList.toggle("a
+   const creating = mode === "create";
+   $("account-confirm-row").classList.toggle("hidden", !creating);
+   $("account-username-hint").classList.toggle("hidden", !creating);
+   $("account-password").setAttribute("autocomplete", creating ? "new-password" :
+   $("btn-account-submit").textContent = TM_I18N.t(creating ? "account.createSubm
+   $("btn-account-submit").setAttribute("data-i18n", creating ? "account.createSu
+   hideError("account-error");
+ }
+
+function openAccountScreen(fromScreen) {
+   $("account-back").dataset.back = fromScreen;
+   hideError("account-error");

```

```

+   $("account-status").classList.add("hidden");
+   $("account-password").value = "";
+   $("account-password2").value = "";
+   $("account-delete-password").value = "";
+   updateAccountUi();
+   showScreen("screen-account");
+}
+
+function accountSnapshot() {
+   return { currency: currentCurrency, language: TM_I18N.getLanguage(), watchlist
+}
+
+// Pull: the server's values win for currency and language; watchlists are
+// merged so stars set before signing in aren't lost. Returns true if the
+// merged result differs from what the server had (so we push it back).
+async function applyRemoteSettings(remote) {
+   if (!remote) return true; // nothing stored yet -- push what this device has
+   accountApplying = true;
+   let serverMissedSomething = false;
+   try {
+      if (remote.currency && TM_PRICES.SUPPORTED_CURRENCIES[remote.currency] && re
+      currentCurrency = remote.currency;
+      chrome.storage.local.set({ [TM_CURRENCY_KEY]: currentCurrency });
+      scheduleAccountPush();
+      const sel = $("currency-select-settings");
+      if (sel) sel.value = currentCurrency;
+   }
+   if (remote.language && remote.language !== TM_I18N.getLanguage()) {
+      TM_I18N.setLanguage(remote.language);
+      document.querySelectorAll(".language-select").forEach((sel) => { sel.value
+   }
+   const remoteList = Array.isArray(remote.watchlist) ? remote.watchlist : [];
+   const before = watchlistSymbols.size;
+   remoteList.forEach((sym) => watchlistSymbols.add(String(sym).toUpperCase()))
+   if (watchlistSymbols.size !== before) {
+      chrome.storage.local.set({ [TM_WATCHLIST_KEY]: Array.from(watchlistSymbols
+   }
+   serverMissedSomething = Array.from(watchlistSymbols).some((s) => !remoteList
+   // Re-render anything that shows prices or translated text.
+   try {
+      if (!$("screen-prices").classList.contains("hidden")) refreshPrices();
+      refreshMainPricesCard();
+      if (currentStatus && currentStatus.unlocked) { refreshBalanceUsd(); refres
+   } catch (e) { /* best effort */ }
+ } finally {
+   accountApplying = false;
+ }

```

```

+   return serverMissedSomething;
+}
+
+function scheduleAccountPush() {
+  if (!accountUser || accountApplying) return;
+  clearTimeout(accountPushTimer);
+  accountPushTimer = setTimeout(pushAccountSettings, 1500);
+}
+
+async function pushAccountSettings() {
+  if (!accountUser) return;
+  try {
+    await accountApi("PUT", "/api/account/settings", { settings: accountSnapshot
+  } catch (e) {
+    if (e.status === 401) { accountUser = null; updateAccountUi(); } // session
+  }
+}
+
+async function onAccountSignedIn(resp) {
+  accountUser = resp.username;
+  const needsPush = await applyRemoteSettings(resp.settings);
+  if (needsPush) await pushAccountSettings();
+  updateAccountUi();
+}
+
+$("#btn-goto-account").addEventListener("click", () => openAccountScreen("screen-
+$("#btn-onboarding-signin").addEventListener("click", () => openAccountScreen("sc
+document.querySelectorAll(".account-tab").forEach((b) => b.addEventListener("cli
+
+$("#btn-account-submit").addEventListener("click", async () => {
+  hideError("account-error");
+  $("account-status").classList.add("hidden");
+  const username = $("account-username").value.trim().toLowerCase();
+  const password = $("account-password").value;
+  if (!/^[a-z0-9_]{3,24}$/.test(username)) return showError("account-error", TM_
+  if (accountMode === "create") {
+    if (password.length < 10) return showError("account-error", TM_I18N.t("accou
+    if (password !== $("account-password2").value) return showError("account-err
+  } else if (!password) {
+    return showError("account-error", TM_I18N.t("account.errBadLogin"));
+  }
+  const btn = $("btn-account-submit");
+  btn.disabled = true;
+  $("account-status").textContent = TM_I18N.t("account.working");
+  $("account-status").classList.remove("hidden");
+  try {
+    const authKey = await deriveAccountAuthKey(username, password);

```

```

+   const resp = await accountApi("POST", accountMode === "create" ? "/api/accou
+   $("account-password").value = "";
+   $("account-password2").value = "";
+   await onAccountSignedIn(resp);
+   $("account-status").textContent = TM_I18N.t("account.synced");
+ } catch (e) {
+   $("account-status").classList.add("hidden");
+   showError("account-error", accountErrorText(e.code));
+ } finally {
+   btn.disabled = false;
+ }
+});
+
+
+$(".btn-account-signout").addEventListener("click", async () => {
+  try { await accountApi("POST", "/api/account/logout", {}); } catch (e) { /* si
+  accountUser = null;
+  clearTimeout(accountPushTimer);
+  updateAccountUi();
+  setAccountMode("signin");
+});
+
+
+$(".btn-account-delete").addEventListener("click", async () => {
+  hideError("account-error");
+  const password = $("account-delete-password").value;
+  if (!password) return showError("account-error", TM_I18N.t("account.deleteNeed
+  if (!window.confirm(TM_I18N.t("account.deleteConfirm"))) return;
+  try {
+    const authKey = await deriveAccountAuthKey(accountUser, password);
+    await accountApi("POST", "/api/account/delete", { authKey });
+    accountUser = null;
+    clearTimeout(accountPushTimer);
+    $("account-delete-password").value = "";
+    updateAccountUi();
+    setAccountMode("signin");
+  } catch (e) {
+    showError("account-error", accountErrorText(e.code));
+  }
+});
+
+
+// Keep the server copy in step whenever one of the synced settings changes.
+document.addEventListener("tm-language-changed", () => { updateAccountUi(); sche
+
+
+async function initAccount() {
+  try {
+    const res = await fetch("/api/account/status", { credentials: "same-origin"
+    const st = await res.json();
+    accountEnabled = !! (st && st.enabled);

```

```

+   } catch (e) {
+     accountEnabled = false; // no API here (older server, file://, offline) -- f
+   }
+   updateAccountUi();
+   if (!accountEnabled) return;
+   try {
+     const me = await accountApi("GET", "/api/account/me");
+     accountUser = me.username;
+     const needsPush = await applyRemoteSettings(me.settings);
+     if (needsPush) scheduleAccountPush();
+   } catch (e) {
+     accountUser = null;
+   }
+   updateAccountUi();
+}
+
// Compact live-prices card on the main screen -- just the first handful of
// PRICE_BOARD's coins, at a glance, without navigating away. Best-effort
// like the balance/token refreshes above: a rate-limited or unreachable
@@ -877,13 +1503,14 @@
  });

// ----- SWAP
-function setupSwapScreen() {
+function setupSwapScreen(pre) {
  hideError("swap-error");
  $("swap-status").classList.add("hidden");
  $("swap-quote-display").classList.add("hidden");
  const unsupported = !currentNetwork.swapRouter;
  $("swap-unsupported").classList.toggle("hidden", !unsupported);
  $("swap-form").classList.toggle("hidden", unsupported);
+ if (!unsupported) populateSwapSelects(pre || {}).catch(() => {});
}

$("btn-swap-quote").addEventListener("click", async () => {
@@ -1163,12 +1790,18 @@
function populateCurrencySelect() {
  const sel = $("currency-select-settings");
  sel.innerHTML = "";
+ const groups = { fiat: document.createElement("optgroup"), crypto: document.cr
+ groups.fiat.label = "Currencies";
+ groups.crypto.label = "Crypto";
  Object.keys(TM_PRICES.SUPPORTED_CURRENCIES).forEach((code) => {
+   const info = TM_PRICES.SUPPORTED_CURRENCIES[code];
+   const opt = document.createElement("option");
+   opt.value = code;
-   opt.textContent = `${TM_PRICES.SUPPORTED_CURRENCIES[code].symbol} ${TM_PRICE

```

```

-   sel.appendChild(opt);
+   opt.textContent = `${info.label} - ${info.name} (${info.symbol})`;
+   (groups[info.type] || groups.fiat).appendChild(opt);
  });
+ sel.appendChild(groups.fiat);
+ sel.appendChild(groups.crypto);
  sel.value = currentCurrency;
  sel.addEventListener("change", async (e) => {
    currentCurrency = e.target.value;
@@ -1540,6 +2173,7 @@
    populateLanguageSelects();
    await loadWatchlist();
    await loadCurrency();
+   initAccount(); // not awaited; see EXTENSION NOTE above -- stays inert here
    populateCurrencySelect();

    const params = new URLSearchParams(location.search);
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/price
--- /tmp/orig/toKen-exchange-main/prices.js      2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/prices.js                2026-09-19 16:54:08.993541121 +0000
@@ -40,6 +40,62 @@
    LTC: "litecoin",
    SHIB: "shiba-inu",
    TON: "the-open-network", // NOT "toncoin" -- CoinGecko's own quirk, verified o
+
+ // ---- Extra coins (added later; see EXTRA_PRICE_BOARD below) ----
+ // These follow CoinGecko's usual id conventions but were NOT each checked
+ // against a live coin page the way the 18 above were, so they are flagged
+ // `optional` on the board: if CoinGecko doesn't return one of them (a wrong
+ // or retired id), that row is quietly left out instead of showing "n/a".
+ BCH: "bitcoin-cash",
+ XLM: "stellar",
+ XMR: "monero",
+ ATOM: "cosmos",
+ NEAR: "near",
+ APT: "aptos",
+ ARB: "arbitrum",
+ OP: "optimism",
+ SUI: "sui",
+ ICP: "internet-computer",
+ FIL: "filecoin",
+ HBAR: "hedera-hashgraph",
+ VET: "vechain",
+ ALGO: "algorand",
+ AAVE: "aave",
+ MKR: "maker",
+ GRT: "the-graph",

```

```

+ INJ: "injective-protocol",
+ RENDER: "render-token",
+ PEPE: "pepe",
+ DAI: "dai",
+ WBTC: "wrapped-bitcoin",
+ ETC: "ethereum-classic",
+ KAS: "kasper",
+ XTZ: "tezos",
+ EOS: "eos",
+ BONK: "bonk",
+ FLOKI: "floki",
+ SAND: "the-sandbox",
+ MANA: "decentraland",
+ LDO: "lido-dao",
+ CRV: "curve-dao-token",
+ STX: "blockstack",
+ CRO: "crypto-com-chain",
+ WIF: "dogwifcoin",
+ TAO: "bittensor",
+ WLD: "worldcoin-wld",
+ PAXG: "pax-gold",
+ TIA: "celestia",
+ SEI: "sei-network",
+ MNT: "mantle",
+ PYTH: "pyth-network",
+ ONDO: "ondo-finance",
+ FET: "fetch-ai",
+ THETA: "theta-token",
+ FLOW: "flow",
+ GALA: "gala",
+ CHZ: "chiliz",
+ COMP: "compound-governance-token",
+ SNX: "havven",
};

```

```

// Maps this wallet's internal network `key` (see lib/networks.js) to the
@@ -83,22 +139,128 @@
  { symbol: "TON", name: "Toncoin" },
];

```

```

+// Extra coins shown on the Prices screen (searchable). `optional: true` means
+// "leave this row out if CoinGecko doesn't return it" -- see the note above.
+const EXTRA_PRICE_BOARD = [
+  ["BCH", "Bitcoin Cash"], ["XLM", "Stellar"], ["XMR", "Monero"], ["ATOM", "Cosm
+  ["NEAR", "NEAR Protocol"], ["APT", "Aptos"], ["ARB", "Arbitrum"], ["OP", "Opti
+  ["SUI", "Sui"], ["ICP", "Internet Computer"], ["FIL", "Filecoin"], ["HBAR", "H
+  ["VET", "VeChain"], ["ALGO", "Algorand"], ["AAVE", "Aave"], ["MKR", "Maker"],

```

```

+ ["GRT", "The Graph"], ["INJ", "Injective"], ["RENDER", "Render"], ["PEPE", "Pe
+ ["DAI", "Dai"], ["WBTC", "Wrapped Bitcoin"], ["ETC", "Ethereum Classic"], ["KA
+ ["XTZ", "Tezos"], ["EOS", "EOS"], ["BONK", "Bonk"], ["FLOKI", "Floki"],
+ ["SAND", "The Sandbox"], ["MANA", "Decentraland"], ["LDO", "Lido DAO"], ["CRV"
+ ["STX", "Stacks"], ["CRO", "Cronos"], ["WIF", "dogwifhat"], ["TAO", "Bittensor
+ ["WLD", "Worldcoin"], ["PAXG", "PAX Gold"], ["TIA", "Celestia"], ["SEI", "Sei"
+ ["MNT", "Mantle"], ["PYTH", "Pyth Network"], ["ONDO", "Ondo"], ["FET", "Fetch.
+ ["THETA", "Theta Network"], ["FLOW", "Flow"], ["GALA", "Gala"], ["CHZ", "Chili
+ ["COMP", "Compound"], ["SNX", "Synthetix"],
+].map(([symbol, name]) => ({ symbol, name, optional: true }));
+
+const FULL_PRICE_BOARD = PRICE_BOARD.concat(EXTRA_PRICE_BOARD);
+
+ // Fiat currencies this wallet offers in Settings, all of which CoinGecko's
+ // keyless API can price directly (no separate FX-rate lookup needed) -- see
+ // https://docs.coingecko.com/reference/simple-supported-vs-currencies.
+ // Symbol is just for display; the code is what's actually sent to CoinGecko.
+const SUPPORTED_CURRENCIES = {
-  usd: { symbol: "$", label: "USD" },
-  eur: { symbol: "€", label: "EUR" },
-  gbp: { symbol: "£", label: "GBP" },
-  jpy: { symbol: "¥", label: "JPY" },
-  cad: { symbol: "CA$", label: "CAD" },
-  aud: { symbol: "AU$", label: "AUD" },
-  inr: { symbol: "₹", label: "INR" },
-  brl: { symbol: "R$", label: "BRL" },
+ // ---- Fiat (national currencies). Every code below is in CoinGecko's own
+ // /simple/supported_vs_currencies list (checked Sept 2026). `decimals`
+ // defaults to 2; currencies with no minor unit in everyday use show 0,
+ // and the Gulf dinars that split into 1000ths show 3.
+ usd: { symbol: "$", label: "USD", name: "US Dollar", type: "fiat" },
+ eur: { symbol: "€", label: "EUR", name: "Euro", type: "fiat" },
+ gbp: { symbol: "£", label: "GBP", name: "British Pound", type: "fiat" },
+ jpy: { symbol: "¥", label: "JPY", name: "Japanese Yen", type: "fiat", decimals
+ cad: { symbol: "CA$", label: "CAD", name: "Canadian Dollar", type: "fiat" },
+ aud: { symbol: "AU$", label: "AUD", name: "Australian Dollar", type: "fiat" },
+ inr: { symbol: "₹", label: "INR", name: "Indian Rupee", type: "fiat" },
+ brl: { symbol: "R$", label: "BRL", name: "Brazilian Real", type: "fiat" },
+ aed: { symbol: "AED", label: "AED", name: "UAE Dirham", type: "fiat" },
+ ars: { symbol: "AR$", label: "ARS", name: "Argentine Peso", type: "fiat" },
+ bdt: { symbol: "৳", label: "BDT", name: "Bangladeshi Taka", type: "fiat" },
+ bhd: { symbol: "BHD", label: "BHD", name: "Bahraini Dinar", type: "fiat", deci
+ bmd: { symbol: "BD$", label: "BMD", name: "Bermudian Dollar", type: "fiat" },
+ chf: { symbol: "CHF", label: "CHF", name: "Swiss Franc", type: "fiat" },
+ clp: { symbol: "CL$", label: "CLP", name: "Chilean Peso", type: "fiat", decima
+ cny: { symbol: "CN¥", label: "CNY", name: "Chinese Yuan", type: "fiat" },
+ czk: { symbol: "Kč", label: "CZK", name: "Czech Koruna", type: "fiat" },

```

```

+ dkk: { symbol: "kr", label: "DKK", name: "Danish Krone", type: "fiat" },
+ gel: { symbol: "ლ", label: "GEL", name: "Georgian Lari", type: "fiat" },
+ hkd: { symbol: "HK$", label: "HKD", name: "Hong Kong Dollar", type: "fiat" },
+ huf: { symbol: "Ft", label: "HUF", name: "Hungarian Forint", type: "fiat", dec
+ idr: { symbol: "Rp", label: "IDR", name: "Indonesian Rupiah", type: "fiat", de
+ ils: { symbol: "₪", label: "ILS", name: "Israeli Shekel", type: "fiat" },
+ krw: { symbol: "₩", label: "KRW", name: "South Korean Won", type: "fiat", deci
+ kwd: { symbol: "KWD", label: "KWD", name: "Kuwaiti Dinar", type: "fiat", decim
+ lkr: { symbol: "Rs", label: "LKR", name: "Sri Lankan Rupee", type: "fiat" },
+ mmk: { symbol: "K", label: "MMK", name: "Myanmar Kyat", type: "fiat", decimals
+ mxn: { symbol: "MX$", label: "MXN", name: "Mexican Peso", type: "fiat" },
+ myr: { symbol: "RM", label: "MYR", name: "Malaysian Ringgit", type: "fiat" },
+ ngn: { symbol: "₦", label: "NGN", name: "Nigerian Naira", type: "fiat" },
+ nok: { symbol: "kr", label: "NOK", name: "Norwegian Krone", type: "fiat" },
+ nzd: { symbol: "NZ$", label: "NZD", name: "New Zealand Dollar", type: "fiat" }
+ php: { symbol: "₱", label: "PHP", name: "Philippine Peso", type: "fiat" },
+ pkr: { symbol: "Rs", label: "PKR", name: "Pakistani Rupee", type: "fiat" },
+ pln: { symbol: "zł", label: "PLN", name: "Polish Zloty", type: "fiat" },
+ rub: { symbol: "₽", label: "RUB", name: "Russian Ruble", type: "fiat" },
+ sar: { symbol: "SAR", label: "SAR", name: "Saudi Riyal", type: "fiat" },
+ sek: { symbol: "kr", label: "SEK", name: "Swedish Krona", type: "fiat" },
+ sgd: { symbol: "S$", label: "SGD", name: "Singapore Dollar", type: "fiat" },
+ thb: { symbol: "฿", label: "THB", name: "Thai Baht", type: "fiat" },
+ try: { symbol: "₺", label: "TRY", name: "Turkish Lira", type: "fiat" },
+ twd: { symbol: "NT$", label: "TWD", name: "New Taiwan Dollar", type: "fiat" },
+ uah: { symbol: "₴", label: "UAH", name: "Ukrainian Hryvnia", type: "fiat" },
+ vnd: { symbol: "₫", label: "VND", name: "Vietnamese Dong", type: "fiat", decim
+ zar: { symbol: "R", label: "ZAR", name: "South African Rand", type: "fiat" },
+
+ // ---- Crypto (also valid CoinGecko vs_currencies, so balances and prices
+ // can be shown in them directly). Shown with significant digits rather
+ // than fixed decimals -- see formatMoney().
+ btc: { symbol: "฿", label: "BTC", name: "Bitcoin", type: "crypto" },
+ eth: { symbol: "Ξ", label: "ETH", name: "Ethereum", type: "crypto" },
+ bnb: { symbol: "BNB", label: "BNB", name: "BNB", type: "crypto" },
+ sol: { symbol: "SOL", label: "SOL", name: "Solana", type: "crypto" },
+ xrp: { symbol: "XRP", label: "XRP", name: "XRP", type: "crypto" },
+ };
+
+ const DEFAULT_CURRENCY = "usd";

+// Compact form for big numbers (market cap, volume): $1.2T, CHF 340B, 12.5M.
+function formatMoneyCompact(amount, currency) {
+  const info = SUPPORTED_CURRENCIES[currency] || SUPPORTED_CURRENCIES[DEFAULT_CU
+  const symbol = /^[A-Za-z]{2,}$/ .test(info.symbol) ? info.symbol + "\u00A0" : i
+  const n = Number(amount);
+  if (!isFinite(n)) return symbol + "0";
+  return symbol + n.toLocaleString(undefined, { notation: "compact", maximumFrac

```

```

+}
+
+// Formats an amount in the given display currency code. `opts.price` is for
+// per-unit coin prices, which keep extra precision under 1 (a $0.0000123
+// meme coin shouldn't read as $0.00). Alphabetic symbols ("CHF", "SAR") get
+// a space before the number; symbol-style ones ("$", "€") don't.
+function formatMoney(amount, currency, opts) {
+  const info = SUPPORTED_CURRENCIES[currency] || SUPPORTED_CURRENCIES[DEFAULT_CU
+  const symbol = /^[A-Za-z]{2,}$/.test(info.symbol) ? info.symbol + "\u00A0" : i
+  const n = Number(amount);
+  if (!isFinite(n)) return symbol + "0";
+  let body;
+  if (info.type === "crypto") {
+    body = n.toLocaleString(undefined, { maximumSignificantDigits: 6 });
+  } else {
+    const base = typeof info.decimals === "number" ? info.decimals : 2;
+    const isPrice = opts && opts.price;
+    if (isPrice && n !== 0 && Math.abs(n) < 0.01) {
+      // Sub-cent unit prices (PEPE, SHIB, BONK...) need significant digits,
+      // not fixed decimals, or they collapse to 0.0000.
+      body = n.toLocaleString(undefined, { maximumSignificantDigits: 4 });
+    } else {
+      const digits = isPrice && Math.abs(n) < 1 ? Math.max(base, 4) : base;
+      body = n.toLocaleString(undefined, { minimumFractionDigits: digits, maximu
+    }
+  }
+  return symbol + body;
+}
+
+// Small in-memory cache. CoinGecko's keyless tier is IP-rate-limited to
+// roughly 10-30 requests/minute -- a 45s TTL keeps normal use (opening the
+// popup, switching screens) well under that without the price ever going
@@ -147,7 +309,7 @@
  if (priceBoardCache.data && priceBoardCache.cacheKey === cacheKey && now - pri
    return priceBoardCache.data;
  }
- const url = `${COINGECKO_BASE}/coins/markets?vs_currency=${vsCurrency}&ids=${e
+ const url = `${COINGECKO_BASE}/coins/markets?vs_currency=${vsCurrency}&ids=${e
  let res;
  try {
    res = await fetch(url);
@@ -171,19 +333,162 @@
  // real logo image URL straight from CoinGecko (null for any of these if
  // CoinGecko didn't return that coin, e.g. a request that partially fails).
  async function getPriceBoard(currency) {
-   const ids = PRICE_BOARD.map((c) => COINGECKO_IDS[c.symbol]);
+   const ids = FULL_PRICE_BOARD.map((c) => COINGECKO_IDS[c.symbol]);

```

```

    const byId = await fetchPriceBoardMarkets(ids, currency);
-   return PRICE_BOARD.map((c) => {
+   const rows = [];
+   FULL_PRICE_BOARD.forEach((c) => {
        const id = COINGECKO_IDS[c.symbol];
        const entry = byId[id];
-       return {
+       // An optional (extra) coin CoinGecko didn't return is left out entirely
+       // rather than shown as "n/a" -- see the note on the extra ids above.
+       if (!entry && c.optional) return;
+       rows.push({
            symbol: c.symbol,
            name: c.name,
            price: entry ? entry.current_price : null,
            change24h: entry && typeof entry.price_change_percentage_24h === "number"
            image: entry ? entry.image : null,
-       };
+       // Public CoinGecko page for this coin; the Prices screen links each row to
+       url: `https://www.coingecko.com/en/coins/${encodeURIComponent(id)}`,
+       });
+   });
+   return rows;
+}
+
+// ---- In-app coin detail (the screen a Prices row opens) -----
+// One CoinGecko /coins/{id} call per coin gives price, market stats and a
+// text description; a second /market_chart call draws the price chart.
+// Both are keyless, cached briefly, and only fired when someone opens a coin.
+const coinDetailCache = new Map();
+const coinChartCache = new Map();
+
+function plainTextFromHtml(html) {
+   return String(html || "")
+       .replace(/<(br|\\p|\\li)[^>]*>/gi, " ")
+       .replace(/<[^>]*>/g, "")
+       .replace(/&/g, "&").replace(/"/g, "'').replace(/&#39;/g, "'")
+       .replace(/&lt;/g, "<").replace(/&gt;/g, ">").replace(/&nbsp;/g, " ")
+       .replace(/\\s+/g, " ")
+       .trim();
+}
+
+// First couple of sentences, capped in length -- this is a summary card,
+// not the whole write-up (the CoinGecko link has the rest).
+function shortDescription(text, maxLen) {
+   if (!text) return "";
+   if (text.length <= maxLen) return text;
+   const cut = text.slice(0, maxLen);

```

```

+   const lastStop = Math.max(cut.lastIndexOf(". "), cut.lastIndexOf("! "), cut.la
+   return (lastStop > maxLen * 0.5 ? cut.slice(0, lastStop + 1) : cut.replace(/\s
+}
+
+async function getCoinDetail(symbol, currency) {
+   const id = COINGECKO_IDS[symbol];
+   if (!id) throw new Error("Unknown coin.");
+   const vs = SUPPORTED_CURRENCIES[currency] ? currency : DEFAULT_CURRENCY;
+   const key = id + "|" + vs;
+   const now = Date.now();
+   const hit = coinDetailCache.get(key);
+   if (hit && now - hit.fetchedAt < CACHE_TTL_MS) return hit.data;
+
+   let res;
+   try {
+     res = await fetch(`${COINGECKO_BASE}/coins/${encodeURIComponent(id)}?localiz
+   } catch (e) {
+     throw new Error("Couldn't reach CoinGecko. Check your internet connection.")
+   }
+   if (!res.ok) throw new Error(`CoinGecko request failed (HTTP ${res.status}).`)
+   const json = await res.json();
+   const md = json.market_data || {};
+   const pick = (o) => (o && typeof o[vs] === "number" ? o[vs] : null);
+   const change = md.price_change_percentage_24h_in_currency && typeof md.price_c
+   ? md.price_change_percentage_24h_in_currency[vs]
+   : (typeof md.price_change_percentage_24h === "number" ? md.price_change_perc
+   const homepage = json.links && Array.isArray(json.links.homepage)
+   ? json.links.homepage.find((u) => typeof u === "string" && /^https:\/\/\//i.te
+   : null;
+   const data = {
+     id,
+     rank: typeof json.market_cap_rank === "number" ? json.market_cap_rank : null
+     price: pick(md.current_price),
+     change24h: change,
+     marketCap: pick(md.market_cap),
+     volume: pick(md.total_volume),
+     high24h: pick(md.high_24h),
+     low24h: pick(md.low_24h),
+     ath: pick(md.ath),
+     athChange: pick(md.ath_change_percentage),
+     circulatingSupply: typeof md.circulating_supply === "number" ? md.circulatin
+     description: shortDescription(plainTextFromHtml(json.description && json.des
+     homepage,
+   };
+   coinDetailCache.set(key, { fetchedAt: now, data });
+   return data;
+}

```

```

+
+// days: 1 (24H), 7, 30, or 365. Returns [[timestampMs, price], ...] thinned
+// to at most ~120 points, plenty for a phone-width line chart.
+async function getCoinChart(symbol, currency, days) {
+  const id = COINGECKO_IDS[symbol];
+  if (!id) throw new Error("Unknown coin.");
+  const vs = SUPPORTED_CURRENCIES[currency] ? currency : DEFAULT_CURRENCY;
+  const key = id + "|" + vs + "|" + days;
+  const now = Date.now();
+  const hit = coinChartCache.get(key);
+  if (hit && now - hit.fetchedAt < CACHE_TTL_MS) return hit.data;
+
+  let res;
+  try {
+    res = await fetch(`${COINGECKO_BASE}/coins/${encodeURIComponent(id)}/market_
+  } catch (e) {
+    throw new Error("Couldn't reach CoinGecko. Check your internet connection.")
+  }
+  if (!res.ok) throw new Error(`CoinGecko chart request failed (HTTP ${res.statu
+  const json = await res.json();
+  const raw = Array.isArray(json.prices) ? json.prices.filter((p) => Array.isArr
+  const step = Math.max(1, Math.ceil(raw.length / 120));
+  const data = raw.filter((_, i) => i % step === 0 || i === raw.length - 1);
+  coinChartCache.set(key, { fetchedAt: now, data });
+  return data;
+}
+
+// Fiat exchange rates: what 1 unit of each supported national currency is
+// worth in the given display currency. Uses CoinGecko's keyless
+// /exchange_rates endpoint, which prices everything against 1 BTC, so any
+// pair is just a ratio: 1 X = rates[display].value / rates[X].value.
+// Purely informational, same as the coin prices. Cached like the others.
+let fiatRatesCache = { fetchedAt: 0, data: null };
+
+async function fetchExchangeRates() {
+  const now = Date.now();
+  if (fiatRatesCache.data && now - fiatRatesCache.fetchedAt < CACHE_TTL_MS) retu
+  let res;
+  try {
+    res = await fetch(`${COINGECKO_BASE}/exchange_rates`);
+  } catch (e) {
+    throw new Error("Couldn't reach CoinGecko for currency rates. Check your int
+  }
+  if (!res.ok) throw new Error(`CoinGecko exchange-rate request failed (HTTP ${r
+  const json = await res.json();
+  const rates = (json && json.rates) || {};
+  fiatRatesCache = { fetchedAt: now, data: rates };

```

```

+   return rates;
+}
+
+async function getFiatRates(currency) {
+   const display = SUPPORTED_CURRENCIES[currency] ? currency : DEFAULT_CURRENCY;
+   const rates = await fetchExchangeRates();
+   const base = rates[display];
+   if (!base || !base.value) return [];
+   const out = [];
+   Object.keys(SUPPORTED_CURRENCIES).forEach((code) => {
+       const info = SUPPORTED_CURRENCIES[code];
+       if (info.type !== "fiat" || code === display) return;
+       const r = rates[code];
+       if (!r || !r.value) return;
+       out.push({ code: info.label, name: info.name, rate: base.value / r.value });
+   });
+   return out;
+}

```

// Returns the live price of the given network's native coin (in the given

@@ -249,6 +554,11 @@

```

if (typeof self !== "undefined") {
    self.TM_PRICES = {
        getPriceBoard,
+       getFiatRates,
+       getCoinDetail,
+       getCoinChart,
+       formatMoney,
+       formatMoneyCompact,
        getNativePriceForNetwork,
        getTokenPricesByContract,
        COINGECKO_IDS,

```

```

diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/pt.js
--- /tmp/orig/toKen-exchange-main/pt.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/pt.js 2026-09-24 23:22:06.590676314 +0000
@@ -72,6 +72,61 @@

```

```

    "prices.description": "Preços em dólar (USD) da API pública da CoinGecko. Ap
    "prices.loading": "Carregando...",
    "prices.naText": "n/d",
+   "prices.tabCrypto": "Cripto",
+   "prices.tabCurrencies": "Moedas",
+   "prices.searchPlaceholder": "Pesquisar...",
+   "prices.currenciesNote": "Quanto vale 1 unidade de cada moeda na sua moeda d
+   "prices.noResults": "Nenhum resultado.",
+   "prices.viewCoin": "Ver {name}",
+   "coin.swapBtn": "Trocar por {symbol}",
+   "coin.swapHint": "Na próxima tela você escolhe com qual dos seus ativos paga

```

```
+ "coin.swapNeedsWallet": "Desbloqueie sua carteira para trocar.",
+ "coin.swapNoRouter": "A troca ainda não está ativada nesta rede.",
+ "coin.swapUnavailableNetwork": "Ainda não é possível trocar {symbol} na rede
+ "coin.holding": "Você tem {amount} {symbol}",
+ "coin.statsTitle": "Dados de mercado",
+ "coin.aboutTitle": "Sobre",
+ "coin.aboutUnavailable": "Nenhuma descrição disponível no momento.",
+ "coin.marketCap": "Valor de mercado",
+ "coin.volume24h": "Volume 24 h",
+ "coin.high24h": "Máxima 24 h",
+ "coin.low24h": "Mínima 24 h",
+ "coin.supply": "Oferta circulante",
+ "coin.ath": "Máxima histórica",
+ "coin.viewCoinGecko": "Ver no CoinGecko",
+ "coin.loadError": "Não foi possível carregar mais detalhes desta moeda agora
+ "coin.chartUnavailable": "Gráfico indisponível no momento.",
+ "account.title": "Entrar",
+ "account.intro": "Opcional. Uma conta sincroniza sua moeda de exibição, idio
+ "account.tabSignIn": "Entrar",
+ "account.tabCreate": "Criar conta",
+ "account.usernameLabel": "Nome de usuário",
+ "account.usernameHint": "3-24 caracteres: letras, números ou sublinhado.",
+ "account.passwordLabel": "Senha",
+ "account.confirmLabel": "Confirmar senha",
+ "account.walletWarning": "Esta senha não pode recuperar sua carteira. Guarde
+ "account.signInSubmit": "Entrar",
+ "account.createSubmit": "Criar conta",
+ "account.working": "Processando...",
+ "account.synced": "Sincronizado.",
+ "account.signedInAs": "Conectado como {username}",
+ "account.syncNote": "Sincronizando: moeda de exibição, idioma e lista de fav
+ "account.signOut": "Sair",
+ "account.deleteLabel": "Excluir conta",
+ "account.deletePlaceholder": "Senha para confirmar",
+ "account.deleteBtn": "Excluir conta",
+ "account.deleteNeedsPassword": "Digite sua senha para excluir sua conta.",
+ "account.deleteConfirm": "Isso exclui sua conta e suas configurações sincron
+ "account.entryBtn": "Entrar / Criar conta",
+ "account.errUsername": "O nome de usuário deve ter 3-24 caracteres: letras,
+ "account.errPasswordShort": "A senha deve ter pelo menos 10 caracteres.",
+ "account.errMismatch": "As senhas não coincidem.",
+ "account.errBadLogin": "Usuário ou senha incorretos.",
+ "account.errTaken": "Esse nome de usuário já está em uso.",
+ "account.errRateLimited": "Muitas tentativas. Aguarde e tente novamente.",
+ "account.errUnavailable": "As contas não estão disponíveis no momento.",
+ "account.errGeneric": "Algo deu errado. Tente novamente.",
+ "swap.customAddressOption": "Outro token (cole o endereço)...",
```

```
    "buy.title": "Comprar cripto",
    "buy.description1": "A compra é processada pela MoonPay, um serviço terceiri
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/ru.js
--- /tmp/orig/toKen-exchange-main/ru.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/ru.js 2026-09-24 23:22:06.591655812 +0000
@@ -73,6 +73,61 @@
    "prices.description": "Цены в USD берутся из публичного API CoinGecko. Это с
    "prices.loading": "Загрузка...",
    "prices.naText": "н/д",
+   "prices.tabCrypto": "Крипто",
+   "prices.tabCurrencies": "Валюты",
+   "prices.searchPlaceholder": "Поиск...",
+   "prices.currenciesNote": "Стоимость 1 единицы каждой валюты в выбранной вами
+   "prices.noResults": "Ничего не найдено.",
+   "prices.viewCoin": "Открыть {name}",
+   "coin.swapBtn": "Обменять на {symbol}",
+   "coin.swapHint": "На следующем экране вы выберете, каким из своих активов пл
+   "coin.swapNeedsWallet": "Разблокируйте кошелёк, чтобы обменять.",
+   "coin.swapNoRouter": "Обмен в этой сети пока недоступен.",
+   "coin.swapUnavailableNetwork": "Обменять {symbol} в сети {network} пока нель
+   "coin.holding": "У вас {amount} {symbol}",
+   "coin.statsTitle": "Рыночные данные",
+   "coin.aboutTitle": "О монете",
+   "coin.aboutUnavailable": "Описание сейчас недоступно.",
+   "coin.marketCap": "Капитализация",
+   "coin.volume24h": "Объём за 24 ч",
+   "coin.high24h": "Максимум за 24 ч",
+   "coin.low24h": "Минимум за 24 ч",
+   "coin.supply": "В обращении",
+   "coin.ath": "Исторический максимум",
+   "coin.viewCoin Gecko": "Открыть на CoinGecko",
+   "coin.loadError": "Не удалось загрузить подробности об этой монете.",
+   "coin.chartUnavailable": "График сейчас недоступен.",
+   "account.title": "Войти",
+   "account.intro": "Необязательно. Аккаунт синхронизирует валюту отображения,
+   "account.tabSignIn": "Войти",
+   "account.tabCreate": "Создать аккаунт",
+   "account.usernameLabel": "Имя пользователя",
+   "account.usernameHint": "3-24 символа: буквы, цифры или подчёркивание.",
+   "account.passwordLabel": "Пароль",
+   "account.confirmLabel": "Подтвердите пароль",
+   "account.walletWarning": "Этот пароль не может восстановить кошелёк. Храните
+   "account.signInSubmit": "Войти",
+   "account.createSubmit": "Создать аккаунт",
+   "account.working": "Выполняется...",
+   "account.synced": "Синхронизировано.",
```

```

+   "account.signedInAs": "Вход выполнен как {username}",
+   "account.syncNote": "Синхронизация: валюта отображения, язык и список избран
+   "account.signOut": "Выйти",
+   "account.deleteLabel": "Удалить аккаунт",
+   "account.deletePlaceholder": "Пароль для подтверждения",
+   "account.deleteBtn": "Удалить аккаунт",
+   "account.deleteNeedsPassword": "Введите пароль, чтобы удалить аккаунт.",
+   "account.deleteConfirm": "Это удалит ваш аккаунт и синхронизированные настро
+   "account.entryBtn": "Войти / Создать аккаунт",
+   "account.errUsername": "Имя пользователя должно содержать 3-24 символа: букв
+   "account.errPasswordShort": "Пароль должен содержать не менее 10 символов.",
+   "account.errMismatch": "Пароли не совпадают.",
+   "account.errBadLogin": "Неверное имя пользователя или пароль.",
+   "account.errTaken": "Это имя пользователя уже занято.",
+   "account.errRateLimited": "Слишком много попыток. Подождите и повторите попы
+   "account.errUnavailable": "Аккаунты сейчас недоступны.",
+   "account.errGeneric": "Что-то пошло не так. Попробуйте ещё раз.",
+   "swap.customAddressOption": "Другой токен (вставьте адрес)...",

    "buy.title": "Купить криптовалюту",
    "buy.description1": "Покупка осуществляется через MoonPay – независимый стор
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/sell-
--- /tmp/orig/toKen-exchange-main/sell-config.js      2026-09-18 22:50:15.00000
+++ toKen-exchange-main/sell-config.js 2026-09-19 05:34:11.345485240 +0000
@@ -36,6 +36,8 @@
    const MOONPAY_SELL_WIDGET_BASE_URL_SANDBOX = "https://sell-sandbox.moonpay.com/"
    const MOONPAY_SELL_WIDGET_BASE_URL_LIVE = "https://sell.moonpay.com/";

+const MOONPAY_SELL_QUOTE_CURRENCIES = new Set(["usd", "eur", "gbp", "jpy", "cad"
+
    function moonpaySellApiKey() {
        return (self.TM_BUY_CONFIG && self.TM_BUY_CONFIG.MOONPAY_PUBLISHABLE_API_KEY)
    }
@@ -66,7 +68,14 @@
    const baseCurrencyCode =
        self.TM_BUY_CONFIG && self.TM_BUY_CONFIG.NETWORK_MOONPAY_CURRENCY_CODE[netwo
        if (baseCurrencyCode) params.set("baseCurrencyCode", baseCurrencyCode);
-    if (quoteCurrency) params.set("quoteCurrencyCode", quoteCurrency);
+    // Only pass fiat codes MoonPay's sell widget is known to accept here (the
+    // original eight). The wallet's display currency list is much longer now
+    // (any CoinGecko-priced currency, plus crypto), and an unsupported
+    // quoteCurrencyCode could break the widget -- for anything else the
+    // widget's own currency picker takes over.
+    if (quoteCurrency && MOONPAY_SELL_QUOTE_CURRENCIES.has(String(quoteCurrency)).t
+    params.set("quoteCurrencyCode", quoteCurrency);
+    }
    return `${sellWidgetBaseUrl(apiKey)}?${params.toString()}`;

```

```

}

diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/serve
--- /tmp/orig/toKen-exchange-main/server.js      2026-09-18 22:50:15.000000000 +00
+++ toKen-exchange-main/server.js                2026-09-19 17:05:19.352443113 +0000
@@ -4,6 +4,7 @@
    const handler = require("serve-handler");
    const { handleFeatureRequestsApi } = require("./feature-requests-api");
    const { handleMoonpaySignApi } = require("./moonpay-sign-api");
+const { handleAccountsApi } = require("./accounts-api");

    const ROOT = __dirname;
    const PUBLIC_DIR = path.join(ROOT, "public");
@@ -75,6 +76,7 @@
    const server = http.createServer((req, res) => {
      if (handleFeatureRequestsApi(req, res)) return;
      if (handleMoonpaySignApi(req, res)) return;
+if (handleAccountsApi(req, res)) return;
      handler(req, res, { public: PUBLIC_DIR });
    });
    server.listen(PORT, () => console.log("Serving on port " + PORT));
diff -ruN '--exclude=*.png' '--exclude=*.jpg' /tmp/orig/toKen-exchange-main/zh.js
--- /tmp/orig/toKen-exchange-main/zh.js 2026-09-18 22:50:15.000000000 +0000
+++ toKen-exchange-main/zh.js 2026-09-24 23:22:06.588437097 +0000
@@ -74,6 +74,61 @@
    "prices.description": "美元价格来自 CoinGecko 的公共 API, 仅供参考—查询过程中不会发
    "prices.loading": "加载中...",
    "prices.noText": "无数据",
+  "prices.tabCrypto": "加密货币",
+  "prices.tabCurrencies": "法定货币",
+  "prices.searchPlaceholder": "搜索...",
+  "prices.currenciesNote": "每种货币 1 单位按您所选显示货币折算的价值。",
+  "prices.noResults": "没有结果。",
+  "prices.viewCoin": "查看 {name}",
+  "coin.swapBtn": "兑换为 {symbol}",
+  "coin.swapHint": "下一屏您可以选择用哪种持有资产来支付。",
+  "coin.swapNeedsWallet": "请解锁钱包后再兑换。",
+  "coin.swapNoRouter": "此网络暂未启用兑换功能。",
+  "coin.swapUnavailableNetwork": "暂时无法在 {network} 上兑换 {symbol}。它必须是该
+  "coin.holding": "您持有 {amount} {symbol}",
+  "coin.statsTitle": "市场数据",
+  "coin.aboutTitle": "简介",
+  "coin.aboutUnavailable": "暂无简介。",
+  "coin.marketCap": "市值",
+  "coin.volume24h": "24 小时成交量",
+  "coin.high24h": "24 小时最高价",
+  "coin.low24h": "24 小时最低价",

```

```
+ "coin.supply": "流通量",
+ "coin.ath": "历史最高价",
+ "coin.viewCoin Gecko": "在 Coin Gecko 上查看",
+ "coin.loadError": "暂时无法加载该币的更多详情。",
+ "coin.chartUnavailable": "图表暂时不可用。",
+ "account.title": "登录",
+ "account.intro": "可选功能。账户可在设备间同步您的显示货币、语言和自选列表，绝不会存储您
+ "account.tabSignIn": "登录",
+ "account.tabCreate": "创建账户",
+ "account.usernameLabel": "用户名",
+ "account.usernameHint": "3-24 个字符：字母、数字或下划线。",
+ "account.passwordLabel": "密码",
+ "account.confirmLabel": "确认密码",
+ "account.walletWarning": "此密码无法找回您的钱包。请妥善保管恢复短语——这是找回资金的唯
+ "account.signInSubmit": "登录",
+ "account.createSubmit": "创建账户",
+ "account.working": "处理中...",
+ "account.synced": "已同步。",
+ "account.signedInAs": "已登录：{username}",
+ "account.syncNote": "同步内容：显示货币、语言和自选列表。",
+ "account.signOut": "退出登录",
+ "account.deleteLabel": "删除账户",
+ "account.deletePlaceholder": "输入密码以确认",
+ "account.deleteBtn": "删除账户",
+ "account.deleteNeedsPassword": "请输入密码以删除您的账户。",
+ "account.deleteConfirm": "这将删除您的账户及其同步设置，不会影响您的钱包和资金。是否继
+ "account.entryBtn": "登录 / 创建账户",
+ "account.errUsername": "用户名须为 3-24 个字符：字母、数字或下划线。",
+ "account.errPasswordShort": "密码至少需要 10 个字符。",
+ "account.errMismatch": "两次输入的密码不一致。",
+ "account.errBadLogin": "用户名或密码不正确。",
+ "account.errTaken": "该用户名已被使用。",
+ "account.errRateLimited": "尝试次数过多，请稍后再试。",
+ "account.errUnavailable": "账户功能目前不可用。",
+ "account.errGeneric": "出了点问题，请重试。",
+ "swap.customAddressOption": "其他代币（粘贴地址）...",

"buy.title": "购买加密货币",
"buy.description1": "购买功能由独立的第三方服务 MoonPay 提供支持——Token Exchange ↗
```